

assignment2

Lim Excelyynx (34476245)

2026-08-25

Assignment 2

Load tidyverse, lubridate, readxl

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.2.1      ✓ readr      2.2.0
## ✓ forcats    1.0.1      ✓ stringr    1.6.0
## ✓ ggplot2    4.0.3      ✓ tibble     3.3.1
## ✓ lubridate  1.9.5      ✓ tidyr      1.3.2
## ✓ purrr      1.2.2
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
```

```
library(readxl)
```

Read and save Wastes.csv and Year_State_ID.csv

```
waste <- read_csv("data/Wastes.csv")
```

```
## Rows: 36718 Columns: 9
## — Column specification —
## Delimiter: ","
## chr (6): Category, Type, Stream, Fate, Core_Non-core, Description
## dbl (3): Case_ID, Year_State_ID, Tonnes
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
waste
```

Case_ID <dbl>	Year_State_ID <dbl>	Category <chr>	
34658	1001	Biosolids	
34659	1001	Biosolids	
34662	1001	Building and demolition materials	
34663	1001	Building and demolition materials	
34664	1001	Building and demolition materials	
34666	1001	Building and demolition materials	
34667	1001	Building and demolition materials	
34668	1001	Building and demolition materials	
34670	1001	Building and demolition materials	
34671	1001	Building and demolition materials	

1-10 of 10,000 rows | 1-3 of 9 columns

Previous123456...1000Next

```
year_state_id <- read_csv("data/Year_State_ID.csv")
```

```
## Rows: 130 Columns: 4
## — Column specification —————
## Delimiter: ","
## chr (2): Year, State
## dbl (2): ID, Economic_Growth
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
year_state_id
```

ID <dbl>	Year <chr>	State <chr>	Economic_Growth <dbl>
1001	2020-2021	ACT	1.07
1002	2020-2021	Australia	-0.03
1003	2020-2021	NSW	-0.88
1004	2020-2021	NT	-0.49
1005	2020-2021	Qld	-0.10
1006	2020-2021	SA	-1.31
1007	2020-2021	Tas	-0.52

ID	Year	State	Economic_Growth
<dbl>	<chr>	<chr>	<dbl>
1008	2020-2021	Vic	-0.87
1009	2020-2021	WA	-1.02
1010	2019-2020	ACT	1.14

1-10 of 130 rows

Previous 1 2 3 4 5 6 ... 13 Next

Determining how to handling missing value

Prior to conducting analysis, we will determine how to handle missing values by first observing how many missing entries are in the waste dataset.

```
missing_entries <- waste %>%
  filter(if_any(everything(), is.na))

missing_entries
```

Case...	Year_State_ID	Category	Type
<dbl>	<dbl>	<chr>	<chr>
34696	1001	Hazardous wastes	Acids (B)
34697	1001	Hazardous wastes	Acids (B)
34698	1001	Hazardous wastes	Acids (B)
34699	1001	Hazardous wastes	Acids (B)
34700	1001	Hazardous wastes	Acids (B)
34701	1001	Hazardous wastes	Acids (B)
34702	1001	Hazardous wastes	Acids (B)
34703	1001	Hazardous wastes	Acids (B)
34704	1001	Hazardous wastes	Acids (B)
34705	1001	Hazardous wastes	Acids (B)

1-10 of 32 rows | 1-5 of 9 columns

Previous 1 2 3 4 Next

The 32 rows with NA observed:

1. A row of Category "Space debris" has several NA values across Case_ID, Year_State_ID, Stream, Fate, Tonnes, Core_Non-core and Description. Hence, the suitable approach is to drop this case. since it is only a small fraction of the entire dataset (1 entry) with several missing values.

```
# drop entry with Category "Space debris"
waste_drop <- waste %>%
  filter(is.na(Category) | Category != "Space debris")
```

2. The remaining 30 entries only has one variable with NA value. The NAs are small in number and located in few cases and variables. Hence, the suitable approach is to impute these values by replacing with appropriate values by doing analysis and observation as we move on to each task.

Q1. Number of unique “Category” values in Wastes.csv

Data Cleaning Category

Observe unique category.

```
waste_drop %>%
  select(Category) %>%
  distinct(Category)
```

Category

<chr>

Biosolids

Building and demolition materials

Building & demolition materials

Glass

Hazardous wastes

Metals

Organics

Plastics

Textiles, leather & rubber (excl. tyres)

Ash

1-10 of 20 rows

Previous **1** 2 Next

Initially, we observed 20 unique Category, including some concerning (inconsistent spelling, invalid category) and NA values, which will require data cleaning.

Some concerning Category are observed. We need to make the Category naming consistent and reassign an appropriate value for miscategorized Category.

```
# record a list of concerning values
concerning_category <- c(
  "Building & demolition materials",
  "Toxic wastes",
  "Plastic",
  "organics",
  "Aluminium",
  "Metals & Organics",
  "Toxic wastes",
  "Recycling"
)
```

To determine how to impute the concerning_category, we can observe others related field like Type, Stream and Fate.

```
# inspect concerning Category values to determine appropriate way on how to impute
waste_drop %>%
  filter(
    Category %in% concerning_category |
    is.na(Category)
  ) %>%
  select(
    Category,
    Type,
    Stream,
    Fate
  )
```

Category
<chr>
Building & demolition materials
Aluminium
Aluminium
Aluminium
organics
organics
Metals & Organics
NA
NA
NA

1-10 of 14 rows | 1-1 of 4 columns

Previous12Next

The concerning Category observed that requires data cleaning action:

1. "Building & demolition materials" should be replaced with "Building and demolition materials" is the same Category but naming inconsistency.
2. "Hazardous wastes" and "HW" is the same Category but naming inconsistency.
3. "Plastic" and "Plastics" is the same Category but naming inconsistency.
4. "organics" and "Organics" is the same Category but naming inconsistency.
5. "Aluminium" is a Type, not Category. Should be replaced with "Metals".
6. "Metals & Organics" entry has Type of "Food organics". Should be replaced with Category of "Organics".
7. "Toxic wastes" has Type mentioning "hazardous wastes". Should be replaced with "Hazardous wastes" for consistency.
8. "Recycling" entry has Type "Polyvinyl chloride (PVC) (3)", which reflects a Category "Plastics". Furthermore, "Recycling" is a Fate, not a Category.

NA Category are further analyse to determine a suitable replacement. We create a na_category_type table to infer waste of similar Type has what kind of category.

```
na_category_type <- waste_drop %>%
  filter(
    is.na(Category)
  ) %>% distinct(Type)

na_category_type
```

Type

<chr>

Other soil/sludges (other N)

Paints, resins, inks, organic sludges (F)

2 rows

Then join na_category_type to waste_drop table to determine a suitable value to replace NA.

```
waste_drop %>% semi_join(
  na_category_type,
  by = "Type"
) %>% select(Category, Type)
```

Category

<chr>

Type

<chr>

Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)

Category <chr>	Type <chr>
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
Hazardous wastes	Other soil/sludges (other N)
1-10 of 2,944 rows	
Previous 1 2 3 4 5 6 ... 295 Next	

It is observed that the all NA Category values with Type of “Other soil/sludges (other N)” and “Paints, resins, inks, organic sludges (F)” belongs to Category of “Hazardous wastes”, informing us:

9. NA entry with Type of “Other soil/sludges (other N)” and “Paints, resins, inks, organic sludges (F)”, which is suitable to be replaced with “Hazardous wastes”.

Now, values of Category column will be updated using `recode()` and `case_when()` accordingly.

```
# impute their correct Category
waste_unique <- waste_drop %>%
  filter(!is.na(Category)) %>%
  mutate(
    Category = recode(
      Category,
      # spelling inconsistency
      "HW" = "Hazardous wastes",
      "Plastic" = "Plastics",
      "organics" = "Organics",
      "Building and demolition materials" = "Building & demolition materials",

      # miscategorisation
      "Aluminium" = "Metals",
      "Metals & Organics" = "Organics",
      "Toxic wastes" = "Hazardous wastes",
      "Recycling" = "Plastics"
    ),
    Category = case_when(
      # missing Category is inferred from Type
      is.na(Category) &
        Type == "Other soil/sludges (other N)" ~
          "Hazardous wastes",
      is.na(Category) &
        Type == "Paints, resins, inks, organic sludges (F)" ~
          "Hazardous wastes",
      TRUE ~ Category
    )
  )
```

Observe new unique Category.

```
waste_unique %>%
  select(Category) %>%
  distinct(Category) %>%
  arrange()
```

Category
<chr>

Biosolids
Building & demolition materials
Glass
Hazardous wastes
Metals
Organics
Plastics
Textiles, leather & rubber (excl. tyres)
Ash
Paper & cardboard

1-10 of 11 rows

Previous12Next

Count unique Category

To count number of unique waste category, we get the number of rows of unique data from Category column in waste_unique table.

```
number_unique_wastes_category <- waste_unique %>%
  summarise(
    Total_Unique_Category = n_distinct(Category)
  )

number_unique_wastes_category
```

Total_Unique_Category
<int>

11

1 row

After data cleaning, the total unique Category has 11 values.

Q2. Number of negative feedback comments in the 'Description' column with an environmental impact score of 2 or 3

The current **waste_unique** as a result from Q1 has its Category value without spelling inconsistency, invalid category or NA value.

Observe Description

Observe the patterns of values of Description column of waste_unique.

```
waste_unique %>%
  select("Description") %>%
  head()
```

Description

<chr>

A total of 20185.55 tonnes of Biosolids (C&I) was sent for Disposal. Environmental Impact Score: 2/10. This approach affects the environment. Rethink disposal methods.

A total of 0.00 tonnes of Biosolids (C&I) was sent for Recycling. Environmental Impact Score: 5/10. You're on the optimizing waste management could help.

A total of 0.00 tonnes of Asphalt (C&D) was sent for Recycling. Environmental Impact Score: 5/10. This has a balance there's room for improvement.

A total of 0.00 tonnes of Asphalt (C&I) was sent for Recycling. Environmental Impact Score: 5/10. Neutral outcome opportunities to reduce waste.

A total of 0.00 tonnes of Asphalt (MSW) was sent for Recycling. Environmental Impact Score: 5/10. Neutral outcome opportunities to reduce waste.

A total of 141663.39 tonnes of Bricks, concrete and pavers (C&D) was sent for Recycling. Environmental Impact making a positive impact on the environment!

6 rows

It is observed that Description values looks like "... Environmental Impact Score: 6/10 ...".

Extract Environmental Impact Score

To obtain environmental impact score, we extract the value from "value/10" of Description column into a new column called Env_Impact_Score.

```

waste_unique <- waste_unique %>%
  mutate(Env_Impact_Score = as.numeric(
    str_extract(
      Description,
      "(?<=Environmental Impact Score: )[0-9]+"
    )
  )
)

waste_unique %>%
  select(Env_Impact_Score) %>%
  arrange(Env_Impact_Score) %>%
  head()

```

Env_Impact_Score
<dbl>

1

1

1

2

2

2

6 rows

To count number of entries with Env_Impact_Score value of 2-3, we filter only rows with Env_Impact_Score = 2 or 3, then mutate a new total_negative_env_impact_score that count the number of rows. We save the this info in number_negative_env_impact_score table.

```

number_negative_env_impact_score <- waste_unique %>%
  filter(
    Env_Impact_Score %in% c(2,3)
  ) %>%
  summarise(
    Total_Negative_Env_Impact_Score = n()
  )

number_negative_env_impact_score

```

Total_Negative_Env_Impact_Score
<int>

6312

1 row

Total negative feedback comments in "Description" with environmental impact score of 2 or 3 is **6312**

Q3. Calculate the fractions of waste tonnes of different waste sources, draw a chart to visualize the fraction numbers specific to the Category value

Data Cleaning Stream

Observe Stream column from waste_unique.

```
waste_unique %>%  
  select(Stream) %>%  
  distinct(Stream)
```

Stream

<chr>

C&I

C&D

MSW

Construction and demolition waste

Municipal solid waste

5 rows

We observed from the Stream column contain duplicated values for the same Stream:

1. "Construction and demolition waste" should be standardized to "C&I"
2. "Municipal solid waste" should be standardized to "MSW"

We will standardized them accordingly to specification: all Stream values will be replace to only 3 options: C&I, C&D, MSW using recode(). This ensure same waste are treated the same stream group when calculating fraction later.

```
waste_unique <- waste_unique %>%  
  mutate(  
    Stream = recode(  
      Stream,  
      "Construction and demolition waste" = "C&D",  
      "Municipal solid waste" = "MSW",  
    )  
  )
```

Observe new unique Stream.

```
waste_unique %>%  
  select(Stream) %>%  
  distinct(Stream)
```

Stream
<chr>
C&I
C&D
MSW
3 rows

Now, Stream is standardized to only 3 valid Stream value.

Data Cleaning Tonnes

Observe any NA Tonnes values from waste_unique.

```
waste_unique %>%
  filter(
    is.na(Tonnes)
  ) %>%
  select(Case_ID, Tonnes, Description)
```

Case_ID	Tonnes
<dbl>	<dbl>
35085	NA
35086	NA
35087	NA
55143	NA
55144	NA
55151	NA
55152	NA
55153	NA
55154	NA
55155	NA
1-10 of 14 rows 1-2 of 3 columns	
Previous 1 2 Next	

We inspect the NA tonnes count and percentage of how much these NA values constitute to overall dataset using summary().

```
# calculate na_tonnes_count and na_percentage
waste_unique %>%
  summarise(
    NA_Tonnes_Count = sum(is.na(Tonnes)),
    Total_Tonnes = n(),
    NA_Percentage = NA_Tonnes_Count / Total_Tonnes * 100
  )
```

NA_Tonnes_Count <int>	Total_Tonnes <int>	NA_Percentage <dbl>
14	36714	0.03813259

1 row

There is a total of 14 NA Tonnes in waste_unique, which makes only a small proportion (0.038%) of the dataset. Furthermore, the other reference column like Description also shows “total of nan tonnes”, which is provide no information on how to impute Tonnes. Hence, NA tonnes will simply be drop

```
# drop NA Tonnes values
waste_unique <- waste_unique %>%
  filter(!is.na(Tonnes))
```

Observe any negative Tonnes values.

```
waste_unique %>%
  select(Case_ID, Tonnes, Description) %>%
  filter(Tonnes < 0)
```

Case_ID <dbl>	Tonnes <dbl>
34666	-2.00e+07
36735	-2.02e-08
36737	-2.02e-10
38955	-2.13e-15
58473	-9.83e-14
62166	-4.98e-14

6 rows | 1-2 of 3 columns

We observed there are 6 values with negative Tonnes values. We need to clean these inputs by observing their respective Description. We observe that:

1. Case_ID of 34666 has a mention of “141663.39 tonnes” in Description. This value should be imputed with 141663.39 tonnes as the Description has provided an obvious corresponding tonnes value.
2. The others have a mention of “-0.00 tonnes” in Description. These values should be imputed with 0 tonnes as we treat them as numerical error. Keeping “-0.00 tonnes” is misleading and unmeaningful.

We will use these observation to impute and replace each negative value using `case_when()`.

```
# replace negative Tonnes values
waste_unique <- waste_unique %>%
  mutate(
    Tonnes = case_when(
      Case_ID == 34666 ~ 141663.39,
      Tonnes < 0 ~ 0,
      TRUE ~ Tonnes
    )
  )
```

Calculating Waste Fraction

To calculate waste fraction, a new `waste_fraction` table is created by:

- Grouping `waste_unique` by `Category` and `Stream`.
- `Stream_Tonnes` is then obtained by summing `Tonnes` in each group.
- `Category_Total` is sum of `Stream_Tonnes`.
- Fraction is `Stream_Tonnes / Category_Total`

```
waste_fraction <- waste_unique %>%
  group_by(Category, Stream) %>%
  summarise(
    Stream_Tonnes = sum(Tonnes, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  group_by(Category) %>%
  mutate(
    Category_Total = sum(Stream_Tonnes),
    Fraction = Stream_Tonnes / Category_Total
  ) %>%
  ungroup() %>%
  mutate(
    Category = factor(
      Category,
      levels = sort(unique(Category))
    )
  )
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Category and Stream.
## i Output is grouped by Category.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Category, Stream))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
waste_fraction %>% arrange(Stream)
```

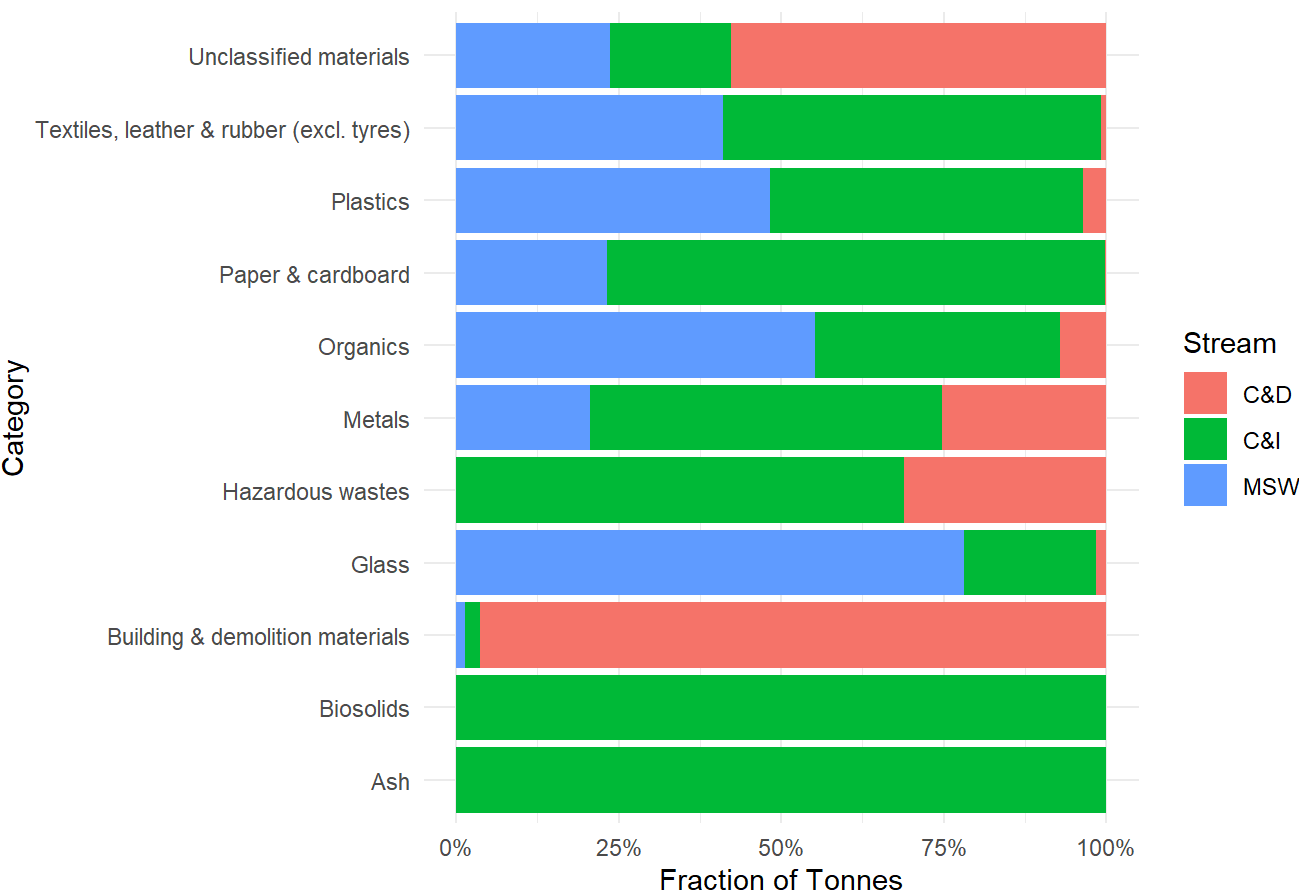
Category <fct>	Stre... <chr>	Stream_Tonnes <dbl>	Category_Total <dbl>	F
Building & demolition materials	C&D	1.216165e+08	126315072.9	0.962
Glass	C&D	1.171805e+05	7130063.4	0.016
Hazardous wastes	C&D	2.800347e+07	90143698.3	0.310
Metals	C&D	9.339725e+06	36947481.4	0.252
Organics	C&D	9.368184e+06	130433955.4	0.071
Paper & cardboard	C&D	4.341863e+04	17228117.5	0.002
Plastics	C&D	3.328685e+04	927040.4	0.035
Textiles, leather & rubber (excl. tyres)	C&D	4.449593e+03	545937.4	0.008
Unclassified materials	C&D	4.492240e+06	7791763.8	0.576
Ash	C&I	1.538592e+08	153859238.0	1.000
1-10 of 28 rows		Previous	1 2 3	Next

Visualization of Fraction of Waste Tonnes by Source of each Category

To visualize the proportional distribution of waste streams within each category, a 100% filled stacked bar chart was selected. The x-axis represents discrete waste Category, y-axis represents the Fraction of Tonnes and fill color distinguishes the contributing waste Streams. A proportional stacked bar chart is suitable because Category and Stream are nominal categorical variables, while the computed fractions represent relative compositions that sum to 100% within each Category. Normalizing the data into proportions allows direct comparison of stream composition across categories of varying total waste volumes.

```
ggplot(
  waste_fraction,
  aes(x = Category, y = Fraction, fill = Stream)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(
    labels = scales::percent_format(),
    limits = c(0, 1)
  ) +
  labs(
    title = "Fraction of Waste Tonnes by Source of each Category",
    x = "Category",
    y = "Fraction of Tonnes",
    fill = "Stream"
  ) +
  theme_minimal()
```

Fraction of Waste Tonnes by Source of each Category



Q4. Add the ‘Year’ and ‘State’ values from Year_State_ID.csv to Wastes.csv, compute the total waste tonnes for each year and state, and store the result in a dataset named ‘temp’, compute and display the statistical information (min, max, mean) of the total waste tonnes for each state in ‘temp’

Data Cleaning Year_State_ID

Inspect the year_state_id unique State values;

```
year_state_id %>% count(State) %>% arrange(n)
```

State	n
<chr>	<int>
Australia	2
ACT	16
NSW	16

State <chr>	n <int>
NT	16
Qld	16
SA	16
Tas	16
Vic	16
WA	16
9 rows	

We observed that:

1. State naming are inconsistent with mixture of upper/lowercase and all upper case.
2. Invalid "Australia" is also observed with only a frequency of 2. As mentioned in brief, State are "State or territory in which the waste was generated." but "Australia" is a country.

```
year_state_id %>% filter(State == "Australia")
```

ID <dbl>	Year <chr>	State <chr>	Economic_Growth <dbl>
1002	2020-2021	Australia	-0.03
1019	2018-2019	Australia	2.30
2 rows			

Inspect the year_state_id unique Year values;

```
year_state_id %>% distinct(Year)
```

Year <chr>
2020-2021
2019-2020
2018-2019
2017-2018
2016-2017
2015-2016
2014-2015
2013-2014

Year

<chr>

2012-2013

2011-2012

1-10 of 23 rows

Previous **1** 2 3 Next

We observe that some Year are invalid financial year (eg: 2005-2007, 2005-2008, 2005-2009 2005-2010, 2005-2011, 2005-2012, 2005-2013) as mentioned in the brief, a valid financial Year should have a range of 1 year (eg: 2006-2007 and 2020-2021).

We will create a new year_clean table that:

1. Upper-cased State for consistency.
2. Remove invalid financial year by separating Year into start and end.
3. Filter only cases where end - start == 1.

```
year_clean <- year_state_id %>%
  # uppercase State
  mutate(
    State = str_to_upper(State)
  ) %>%
  # separate start and end year
  separate(
    Year,
    into = c("start", "end"),
    sep = "-",
    remove = FALSE) %>%
  # convert to numeric
  mutate(
    start = as.numeric(start),
    end = as.numeric(end)
  ) %>%
  # remove invalid year range
  filter(
    end - start == 1
  )
```

year_clean

ID	Year	start	end	State	Economic_Growth
<dbl>	<chr>	<dbl>	<dbl>	<chr>	<dbl>
1001	2020-2021	2020	2021	ACT	1.07
1002	2020-2021	2020	2021	AUSTRALIA	-0.03
1003	2020-2021	2020	2021	NSW	-0.88
1004	2020-2021	2020	2021	NT	-0.49
1005	2020-2021	2020	2021	QLD	-0.10

ID	Year	start	end	State	Economic_Growth
<dbl>	<chr>	<dbl>	<dbl>	<chr>	<dbl>
1006	2020-2021	2020	2021	SA	-1.31
1007	2020-2021	2020	2021	TAS	-0.52
1008	2020-2021	2020	2021	VIC	-0.87
1009	2020-2021	2020	2021	WA	-1.02
1010	2019-2020	2019	2020	ACT	1.14

1-10 of 123 rows

Previous 1 2 3 4 5 6 ... 13 Next

Joining Year_State_ID.csv and Wastes.csv into temp

To combine both Year_State_ID.csv and Wastes.csv into a waste_year table, left join waste_unique as primary table to year_clean as look-up table. Left-join is used because the main goal is to add Year and State attribute from year_clean onto every entry in waste_unique. Left join will link foreign key (Year_State_ID from waste_unique) to primary key (ID in year_clean). This ensures all observation records from waste_unique are fully preserved.

```
waste_year <- waste_unique %>%
  left_join(
    year_clean,
    by = c("Year_State_ID" = "ID")
  )
```

To determine how to handle "AUSTRALIA", we observe the row.

```
waste_year %>% filter(State == "AUSTRALIA")
```

Case...	Year_State_ID	Category	Type	Stre...	Fate
<dbl>	<dbl>	<chr>	<chr>	<chr>	<chr>
35206	1002	Textiles, leather & rubber (excl. tyres)	Textiles	MSW	Recy

1 row | 1-7 of 15 columns

Since there is no useful information on how to impute State of "AUSTRALIA" and considering it is only 1 entry of the entire dataset, we will simply filter to remove the invalid AUSTRALIA state.

```
waste_year <- waste_year %>%
  filter(State != "AUSTRALIA")
```

Temp dataset

Next, we create the new temp by:

- Calculating Tonnes_Waste_Year. Year and State are grouped in order to compute Tonnes_Waste_Year by summarizing to sum Tonnes of each group.

- Reshaping waste_year to make each state as a single column using pivot_wider().

```
temp <- waste_year %>%
  group_by(Year, State) %>%
  summarise(
    Total_Waste_Tonnes = sum(Tonnes, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  pivot_wider(
    names_from = State,
    values_from = Total_Waste_Tonnes
  )
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Year and State.
## i Output is grouped by Year.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Year, State))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

temp

Year <chr>	ACT <dbl>	NSW <dbl>	NT <dbl>	QLD <dbl>	SA <dbl>	TAS <dbl>	VIC <dbl>	WA <dbl>
2006-2007	123348.03	12133529	211021.81	8406112	1133677.4	518567.3	10781626	2845501.3
2007-2008	10254.25	1117131	31372.45	2319276	350233.8	270085.9	1171088	435379.3
2008-2009	132197.33	12788913	160833.53	8436562	1152151.8	473074.7	10026413	2898689.6
2009-2010	106225.47	12120537	156945.30	8124255	1149675.0	511534.8	11466033	3211197.5
2010-2011	129489.45	11647741	149741.53	7921023	1318715.3	548643.2	11889323	2908426.6
2011-2012	15808.47	1279097	33725.89	1182726	866612.2	292449.4	1035186	566392.4
2012-2013	17249.29	1695348	34686.35	1592960	546370.9	292240.8	1118306	619996.9
2013-2014	125560.61	11478818	218152.64	8100391	1247949.8	607127.0	11362007	5276235.7
2014-2015	140262.29	19886566	174951.82	8113019	1298196.4	655359.3	11846287	5269660.8
2015-2016	266898.24	18756824	140152.90	7863865	4330292.6	787983.5	11271973	5211858.5
1-10 of 15 rows							Previous	1 2 Next

Summary for temp

State the summary (min, q1, median, mean, q3, max) of each state in single function using summary().

```
summary(temp)
```

```
##      Year      ACT      NSW      NT
## Length :15 Min. : 10254 Min. : 1117131 Min. : 31372
## N.unique :15 1st Qu.:114787 1st Qu.:11563280 1st Qu.:144947
## N.blank : 0 Median :132197 Median :12788913 Median :164031
## Min.nchar: 9 Mean :261622 Mean :13952194 Mean :153095
## Max.nchar: 9 3rd Qu.:325202 3rd Qu.:20919331 3rd Qu.:203092
##      Max. :939356 Max. :21707179 Max. :247840
##      QLD      SA      TAS      VIC
## Min. : 1182726 Min. : 350234 Min. :270086 Min. : 1035186
## 1st Qu.: 7892444 1st Qu.:1141676 1st Qu.:492305 1st Qu.:10404019
## Median : 8124255 Median :1298196 Median :548643 Median :11466033
## Mean : 8111748 Mean :2317120 Mean :532748 Mean :10142362
## 3rd Qu.:10408971 3rd Qu.:4212757 3rd Qu.:624448 3rd Qu.:12698130
## Max. :13242400 Max. :4641795 Max. :787983 Max. :15468346
##      WA
## Min. : 435379
## 1st Qu.:2872095
## Median :5045902
## Mean :3827433
## 3rd Qu.:5272948
## Max. :6551576
```

Q5. Yearly trend of total waste tonnes of food organics for each state

Format Year

To format the year from, for example: "2020-2021" to "2020", a new `waste_year_formatted` table is from `year_waste` in Q4. Year is extracted by extracting the first 4 digit from original Year column. This ensures `waste_year_formatted` table can be used for remaining task, including visualization of yearly trend of total waste tonnes of food organics for each state later.

```
waste_year_formatted <- waste_year %>%
  mutate(
    Year = as.numeric(str_extract(Year, "^\\d{4}"))
  )

waste_year_formatted %>% head()
```

Case...	Year_State_ID	Category	Type
<dbl>	<dbl>	<chr>	<chr>
34658	1001	Biosolids	Biosolids
34659	1001	Biosolids	Biosolids
34662	1001	Building & demolition materials	Asphalt
34663	1001	Building & demolition materials	Asphalt

Case...	Year_State_ID	Category	Type
<dbl>	<dbl>	<chr>	<chr>
34664	1001	Building & demolition materials	Asphalt
34666	1001	Building & demolition materials	Bricks, concrete and pavers

Observe Type in waste_year_formatted.

```
waste_year_formatted %>%
  count(Type) %>%
  arrange(n)
```

Type
<chr>
Ceramics, tiles and pottery
Other glass
Soil, sand and rock not contaminated above any threshold requiring classification as contaminated soils (N120)
Certified compostable plastics
Low density polyethylene (LDPE) (4)
Polypropylene (PP) (5)
Polystyrene (PS) (6)
Ash
Other plastics (7)
Polyvinyl chloride (PVC) (3)

1-10 of 51 rows | 1-1 of 2 columns

Previous123456Next

We observe that all Type has been assigned to a valid entry and no NA Type is detected. Thus, no data cleaning is required on Type.

Filter food organics

Before analyzing food organics, a new food_organics_waste table with only data of food organics is created. The entries consist of Type that equals “Food Organics”. To obtain Food_Organic_Tonnes, table is grouped by Year and State, then obtained by summing each group Tonnes values.

The food_organics_waste table has: 1. Year 2. State 3. Food_Organics_Tonnes

```
food_organics_waste <- waste_year_formatted %>%
  filter(
    Type == "Food organics"
  ) %>%
  group_by(Year, State) %>%
  summarise(
    Food_Organics_Tonnes = sum(Tonnes, na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Year and State.
## i Output is grouped by Year.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Year, State))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

food_organics_waste

Year	State	Food_Organics_Tonnes
<dbl>	<chr>	<dbl>
2006	ACT	55924.00
2006	NSW	1518923.00
2006	NT	84107.00
2006	QLD	892688.00
2006	SA	247329.00
2006	TAS	132097.00
2006	VIC	1316123.00
2006	WA	651059.00
2008	ACT	46602.00
2008	NSW	1694021.00

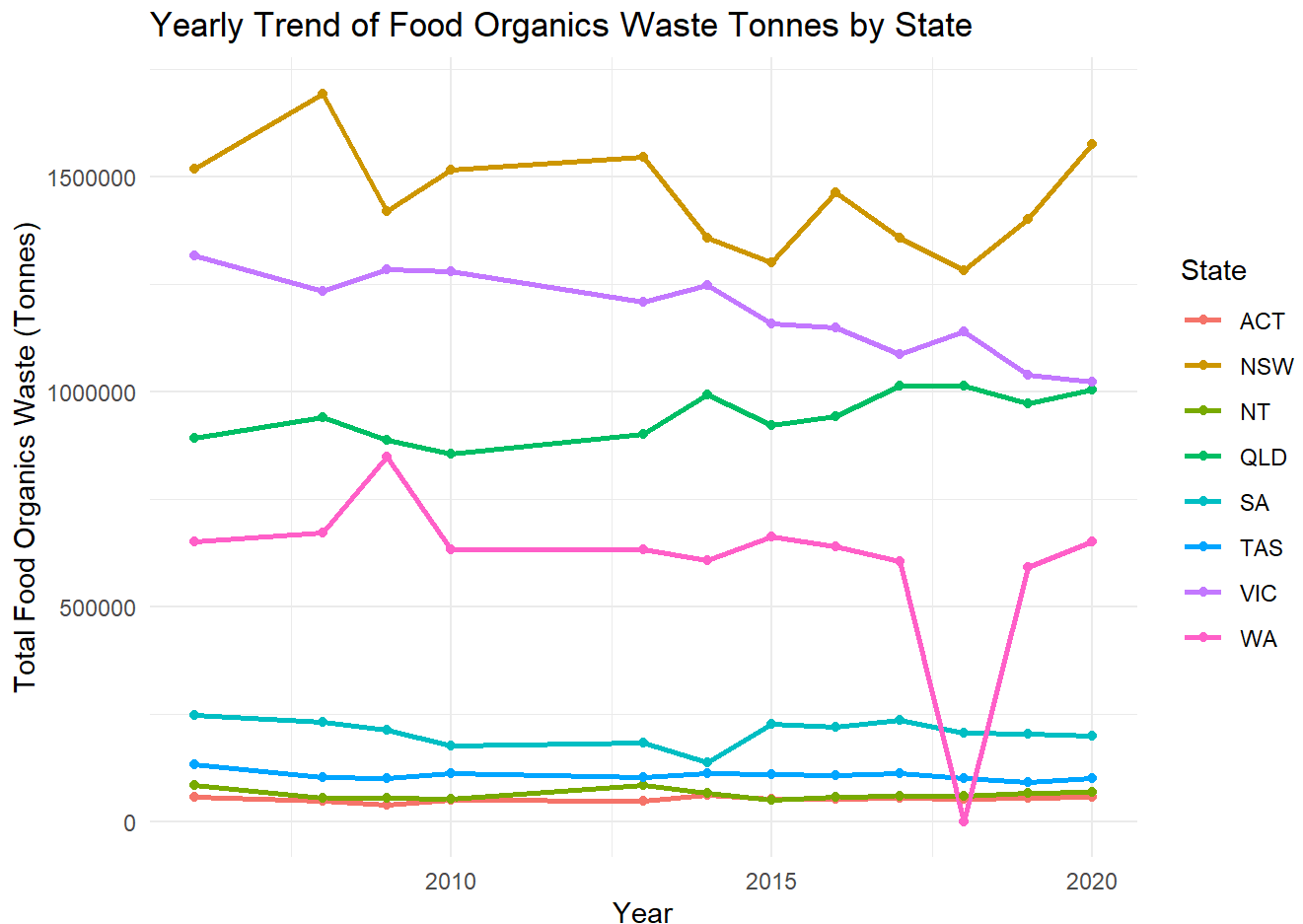
1-10 of 96 rows

Previous 1 2 3 4 5 6 ... 10 Next

Plot food organics by year

To observe yearly trend of food organics waste tonnes by state, a multiline chart is plotted with x-axis as year and y-axis as total food organics waste tonnes, each line is of a different color to represent each state. Line chart is suitable because we want to visualise the trend of food organics waste across a timeline. Line of differing color allow efficient comparative evaluation.

```
ggplot(food_organics_waste, aes(x = Year, y = Food_Organics_Tonnes, color = State)) +
  geom_line(linewidth = 1) +
  geom_point() +
  labs(
    title = "Yearly Trend of Food Organics Waste Tonnes by State",
    x = "Year",
    y = "Total Food Organics Waste (Tonnes)",
    color = "State"
  ) +
  theme_minimal()
```



Q6. Display the most recycled waste Type and the most energy-recovered waste Type with the corresponding year. Display the most increased waste Type over years

Observe Fate values of waste_year_formatted.

```
waste_year_formatted %>% count(Fate) %>% arrange(n)
```

Fate	n
<chr>	<int>
Waste reuse	19
Long-term storage	60
Energy recovery	7570
Recycling	13934
Disposal	15116
5 rows	

We observe that all 5 Fate values are valid, distinct with non-zero count as mentioned in brief without NA values present. Thus, no data cleaning on Fate is required.

Waste with recycling as Fate

A new fate_recycling table is created with entries consist of filtered Fate that equals “Recycling”.

```
fate_recycling <- waste_year_formatted %>%
  filter(Fate == "Recycling")

fate_recycling
```

Case_ID	Year_State_ID	Category	
<dbl>	<dbl>	<chr>	
34659	1001	Biosolids	
34662	1001	Building & demolition materials	
34663	1001	Building & demolition materials	
34664	1001	Building & demolition materials	
34666	1001	Building & demolition materials	
34667	1001	Building & demolition materials	
34668	1001	Building & demolition materials	
34670	1001	Building & demolition materials	
34671	1001	Building & demolition materials	
34672	1001	Building & demolition materials	
1-10 of 10,000 rows 1-3 of 15 columns			
Previous 1 2 3 4 5 6 ... 1000 Next			

Sum per year of each waste type with recycling as Fate

To obtain Recycled_Tonnes, fate_recycling table is grouped by Type and Year, then obtained by summing each group Tonnes values.

```
recycled_waste_type <- fate_recycling %>%
  group_by(Type, Year) %>%
  summarise(
    Recycled_Tonnes = sum(Tonnes, na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Type and Year.
## i Output is grouped by Type.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Type, Year))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
recycled_waste_type %>% arrange(Year)
```

Type <chr>	Year <dbl>	Recycled_Tonnes <dbl>
Acids (B)	2006	4.843852e+04
Alkalis (C)	2006	1.948838e+05
Aluminium	2006	6.380100e+04
Asbestos (N220)	2006	1.273701e+02
Ash	2006	4.276810e+06
Asphalt	2006	1.895200e+05
Biosolids	2006	9.783052e+05
Bricks, concrete and pavers	2006	2.133152e+06
Cardboard	2006	5.092000e+05
Clinical and pharmaceutical (R)	2006	3.983639e+02
1-10 of 628 rows		Previous 1 2 3 4 5 6 ... 63 Next

Most recycled waste type

To obtain the most recycled waste type, we filter entry with its Recycled_Tonnes equating to the max of the Recycled_Tonnes.

```
most_recycled_waste_type <- recycled_waste_type %>%
  filter(Recycled_Tonnes == max(Recycled_Tonnes))

most_recycled_waste_type
```

Type <chr>	Year <dbl>	Recycled_Tonnes <dbl>
Bricks, concrete and pavers	2020	10478681
1 row		

The most recycled waste Type is **Bricks, concrete and pavers** in the year **2020** with a total tonnes of **10478681** tonnes.

Waste with energy-recovered as Fate

A new fate_energy_recovered table is created with entries consist of filtered Fate that equals "Energy Recovered".

```
fate_energy_recovery <- waste_year_formatted %>%
  filter(Fate %in% c("Energy recovery"))

fate_energy_recovery
```

Case... <dbl>	Year_State_ID <dbl>	Category <chr>	Type <chr>
34696	1001	Hazardous wastes	Acids (B)
34701	1001	Hazardous wastes	Acids (B)
34703	1001	Hazardous wastes	Acids (B)
34708	1001	Hazardous wastes	Acids (B)
34717	1001	Hazardous wastes	Alkalis (C)
34722	1001	Hazardous wastes	Alkalis (C)
34724	1001	Hazardous wastes	Alkalis (C)
34729	1001	Hazardous wastes	Alkalis (C)
34738	1001	Hazardous wastes	Asbestos (N220)
34743	1001	Hazardous wastes	Asbestos (N220)
1-10 of 7,570 rows 1-5 of 15 columns			
Previous 1 2 3 4 5 6 ... 757 Next			

Sum per year of each waste type with energy-recovered as Fate

To obtain Energy_Recovered_Tonnes, fate_energy_recovered table is grouped by Type and Year, then obtained by summing each group Tonnes values.

```
energy_recovered_waste_type <- fate_energy_recovery %>%
  group_by(Type, Year) %>%
  summarise(Energy_Recovered_Tonnes = sum(Tonnes, na.rm = TRUE)) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Type and Year.
## i Output is grouped by Type.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Type, Year))` for per-operation grouping
## (`?dplyr::dplyr_by`) instead.
```

```
energy_recovered_waste_type %>% arrange(Year)
```

Type <chr>	Year <dbl>	Energy_Recovered_Tonnes <dbl>
Acids (B)	2006	0.0000
Alkalis (C)	2006	0.0000
Asbestos (N220)	2006	0.0000
Clinical and pharmaceutical (R)	2006	0.0000
Contaminated soils (N120)	2006	0.0000
Food organics	2006	903597.0000
Food-derived hazardous wastes (K100, K110)	2006	0.0000
Garden organics	2006	222580.0000
Inorganic chemicals (D)	2006	0.0000
Oils (J)	2006	53520.0878
1-10 of 333 rows		Previous 1 2 3 4 5 6 ... 34 Next

Most energy-recovered waste type

To obtain the most energy-recovered waste type, we filter entry with its Energy_Recovered_Tonnes equating to the max of the Energy_Recovered_Tonnes

```
most_energy_recovered_waste_type <- energy_recovered_waste_type %>%
  filter(Energy_Recovered_Tonnes == max(Energy_Recovered_Tonnes))

most_energy_recovered_waste_type
```

Type <chr>	Year <dbl>	Energy_Recovered_Tonnes <dbl>
Food organics	2015	1140998
1 row		

The most recycled waste Type is **Food organics** in the year **2015** with a total tonnes of **1140998**.

Find first (2006) and last year (2020) data of each waste type

Filter only entry in waste_year_formatted that has a Tonnes value in the year of 2006 and 2020 as described in brief, the original data presented in Year_State_ID.csv is between 2006-2007 (2006) and 2020-2021 (2020).

```
first_last_waste_type <- waste_year_formatted %>%
  filter(Year %in% c(2006, 2020)) %>%
  distinct(Type, Year) %>%
  count(Type, name = "Count")

first_last_waste_type %>% arrange(Count)
```

Type

<chr>

Ceramics, tiles and pottery

Certified compostable plastics

Other glass

Soil, sand and rock not contaminated above any threshold requiring classification as contaminated soils (N120)

Unclassified materials

Acids (B)

Alkalis (C)

Aluminium

Asbestos (N220)

Ash

1-10 of 51 rows | 1-1 of 2 columns

Previous 1 2 3 4 5 6 Next

To ensure we can calculate most incremented waste type over the year, we need both start (2006) and end (2020) year to compute. We observed that 5 waste types only have either one (2006 or 2020) data, thus would fail to produce a result (end - start) if computation were done. Hence, we need to remove these values before proceeding for computation.

```
# keep only waste type where Count == 2 (contain Tonnes value for both year 2006 and 2020)
first_last_waste_type <- first_last_waste_type %>%
  filter(Count == 2)

first_last_waste_type
```

Type

<chr>

Count

<int>

Acids (B)

2

Type <chr>	Count <int>
Alkalis (C)	2
Aluminium	2
Asbestos (N220)	2
Ash	2
Asphalt	2
Biosolids	2
Bricks, concrete and pavers	2
Cardboard	2
Clinical and pharmaceutical (R)	2
1-10 of 46 rows	Previous 1 2 3 4 5 Next

Create a new `start_end_waste_type` from `waste_year_formatted` where only waste types with Type from `first_last_waste_type` and Year with 2006 and 2016.

Create new `waste_type_increase` from `start_end_waste_type` to have fields for:

- `Total_Tonnes` = sum of Tonnes from 2006 to 2020
- `Start_Tonnes` = Tonnes from year 2006 - `End_Tonnes` = Tonnes from year 2020
- `Change` = `End_Tonnes` - `Start_Tonnes`

```
waste_type_start_end <- waste_year_formatted %>%
  filter(
    Type %in% first_last_waste_type$Type,
    Year %in% c(2006, 2020)
  ) %>%
  group_by(Type, Year) %>%
  summarise(
    Total_Waste_Tonnes = sum(Tonnes, na.rm = TRUE),
  ) %>%
  ungroup() %>%
  group_by(Type) %>%
  summarise(
    Start_Tonnes = Total_Waste_Tonnes[Year == 2006],
    End_Tonnes = Total_Waste_Tonnes[Year == 2020],
    Change = End_Tonnes - Start_Tonnes,
  ) %>%
  ungroup()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Type and Year.
## i Output is grouped by Type.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Type, Year))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

waste_type_start_end

Type <chr>	Start_Tonnes <dbl>	End_Tonnes <dbl>	Change <dbl>
Acids (B)	6.823234e+04	5.022784e+04	-18004.50
Alkalis (C)	2.187616e+05	2.784596e+05	59697.99
Aluminium	6.380100e+04	1.942654e+05	130464.44
Asbestos (N220)	3.156542e+05	1.406474e+06	1090819.43
Ash	1.435095e+07	1.202259e+07	-2328357.99
Asphalt	1.895200e+05	1.693568e+06	1504048.15
Biosolids	1.367297e+06	1.604094e+06	236796.51
Bricks, concrete and pavers	2.133152e+06	1.047868e+07	8345528.72
Cardboard	5.092000e+05	1.290123e+06	780922.55
Clinical and pharmaceutical (R)	6.155006e+04	5.667315e+04	-4876.90

1-10 of 46 rows

Previous12345Next

Most incremented waste type

To obtain the most incremented waste_type, filter all row to obtain Change that equates to highest changes value of the column.

```
most_incremented_waste_type <- waste_type_start_end %>%
  filter(Change == max(Change))

most_incremented_waste_type
```

Type <chr>	Start_Tonnes <dbl>	End_Tonnes <dbl>	Change <dbl>
Bricks, concrete and pavers	2133152	10478681	8345529

1 row

The most incremented waste type over year is **Bricks, concrete and pavers** with an increase of **8345529** Tonnes.

Q7. Investigate the factors influencing environmental impact scores (EIS)

The waste_year_formatted table contain all join data from Wastes.csv and Year_State_ID.csv, with

Data Cleaning Core_Non-core

Year representing the start year (eg: 2020 from 2020-2021). The Core_Non-core column is still unclean up to this point.

Observe NA Core_Non-core value.

```
waste_year_formatted %>% filter(is.na(`Core_Non-core`))
```

Case...	Year_State_ID	Category	Type	Stre...	Fate	Tonnes	Co
<dbl>	<dbl>	<chr>	<chr>	<chr>	<chr>	<dbl>	<c
34696	1001	Hazardous wastes	Acids (B)	C&D	Energy recovery	0.00000000	NA
34697	1001	Hazardous wastes	Acids (B)	C&D	Disposal	0.00000000	NA
34698	1001	Hazardous wastes	Acids (B)	C&D	Disposal	0.00000000	NA
34699	1001	Hazardous wastes	Acids (B)	C&D	Recycling	0.00000000	NA
34700	1001	Hazardous wastes	Acids (B)	C&D	Disposal	0.00000000	NA
34701	1001	Hazardous wastes	Acids (B)	C&D	Energy recovery	0.00000000	NA
34702	1001	Hazardous wastes	Acids (B)	C&D	Recycling	0.00000000	NA
34703	1001	Hazardous wastes	Acids (B)	C&I	Energy recovery	0.00000000	NA
34704	1001	Hazardous wastes	Acids (B)	C&I	Disposal	0.01431044	NA
34705	1001	Hazardous wastes	Acids (B)	C&I	Disposal	0.00001420	NA

1-10 of 14 rows | 1-8 of 15 columns

Previous 1 2 Next

Based on the brief, a core waste comprise of “hazardous waste”. As observed, all entries with NA as Core_Non-core are of Category of “Hazardous waste”. Hence, we can impute these entries with “Core waste” as the Core_Non-core value.

```
waste_year_formatted <- waste_year_formatted %>%
  mutate(
    `Core_Non-core` = case_when(
      is.na(`Core_Non-core`) & Category == "Hazardous wastes" ~ "Non-core waste",
      TRUE ~ `Core_Non-core`
    )
  )
```

Analyse Environmental Impact Score (EIS)

To analyse the factors influencing EIS, we will be using the previous waste_year_formatted dataset.

```
glimpse(waste_year_formatted)

## Rows: 36,699
## Columns: 15
## $ Case_ID      <dbl> 34658, 34659, 34662, 34663, 34664, 34666, 34667, 3466...
## $ Year_State_ID <dbl> 1001, 1001, 1001, 1001, 1001, 1001, 1001, 1001, 1001,...
## $ Category     <chr> "Biosolids", "Biosolids", "Building & demolition mate...
## $ Type         <chr> "Biosolids", "Biosolids", "Asphalt", "Asphalt", "Asph...
## $ Stream       <chr> "C&I", "C&I", "C&D", "C&I", "MSW", "C&D", "C&I", "MSW...
## $ Fate         <chr> "Disposal", "Recycling", "Recycling", "Recycling", "R...
## $ Tonnes       <dbl> 2.018555e+04, 0.000000e+00, 0.000000e+00, 0.000000e+0...
## $ `Core_Non-core` <chr> "Core waste", "Core waste", "Core waste", "Core waste...
## $ Description  <chr> "A total of 20185.55 tonnes of Biosolids (C&I) was se...
## $ Env_Impact_Score <dbl> 2, 5, 5, 5, 5, 8, 8, 5, 8, 8, 5, 8, 8, 5, 8, 8, 5, 8,...
## $ Year        <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020,...
## $ start       <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020,...
## $ end         <dbl> 2021, 2021, 2021, 2021, 2021, 2021, 2021, 2021, 2021,...
## $ State       <chr> "ACT", "ACT", "ACT", "ACT", "ACT", "ACT", "ACT", "ACT", "ACT...
## $ Economic_Growth <dbl> 1.07, 1.07, 1.07, 1.07, 1.07, 1.07, 1.07, 1.07, 1.07,...
```

Environmental Impact Score (EIS) distribution

Firstly, we need to identify EIS is continuous or discrete.

waste_year_formatted %>%
 count(Env_Impact_Score) %>%
 arrange(Env_Impact_Score)

Env_Impact_Score	n
<dbl>	<int>
1	3
2	6
3	6306
4	2
5	16106
6	8799
8	5477

7 rows

```
waste_year_formatted %>% distinct(Env_Impact_Score)
```

Env_Impact_Score
<dbl>
2
5
8
6
3
1
4
7 rows

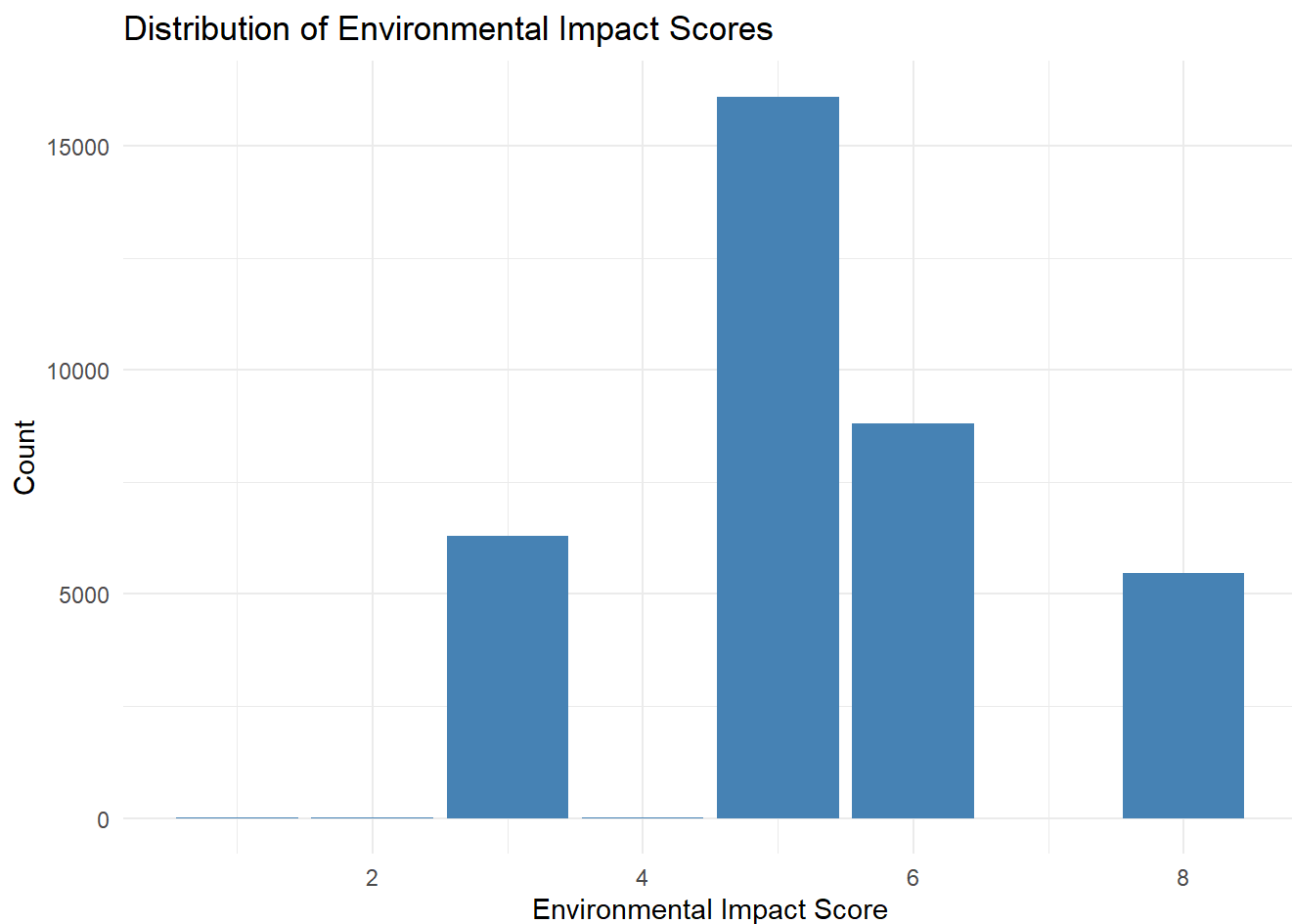
```
waste_year_formatted %>%
  count(Env_Impact_Score) %>%
  arrange(Env_Impact_Score)
```

Env_Impact_Score	n
<dbl>	<int>
1	3
2	6
3	6306
4	2
5	16106
6	8799
8	5477
7 rows	

Here, we observed the EIS found in in dataset are valid as they all fall within range of 1-10, no NA value is observed. The available Env_Impact_Score with entries are 1, 2, 3, 4, 5, 6, 8 – indicating EIS is discrete. EIS are not distributed uniformly across the scale of 1-10. Instead, we observe how EIS is largely concentrating at 5, 6, 3, 8. Whereas the remaining EIS of 1, 2 and 4 only has less than count of 10. The rest of EIS 7, 9, 10 are not observed.

When we visualize EIS distribution using a bar-chart, we can clearly observe EIS is categorical rather than continuous.

```
ggplot(waste_year_formatted, aes(x = Env_Impact_Score)) +  
  geom_bar(fill = "steelblue") +  
  labs(  
    title = "Distribution of Environmental Impact Scores",  
    x = "Environmental Impact Score",  
    y = "Count"  
  ) +  
  theme_minimal()
```



Is Category a factor?

To analyse if Category influence EIS, we first compute the statistical information of EIS for each Category.

```
category_summary_table <- waste_year_formatted %>%
  group_by(Category) %>%
  summarise(
    Mean_EIS = mean(Env_Impact_Score, na.rm = TRUE),
    SD_EIS = sd(Env_Impact_Score, na.rm = TRUE),
    Total_Count = n(),
    Min = min(Env_Impact_Score, na.rm = TRUE),
    First_Quartile = quantile(Env_Impact_Score, 0.25, na.rm = TRUE),
    Median = median(Env_Impact_Score, na.rm = TRUE),
    Third_Quartile = quantile(Env_Impact_Score, 0.75, na.rm = TRUE),
    Max = max(Env_Impact_Score, na.rm = TRUE)
  ) %>% ungroup()

category_summary_table %>% arrange(Mean_EIS)
```

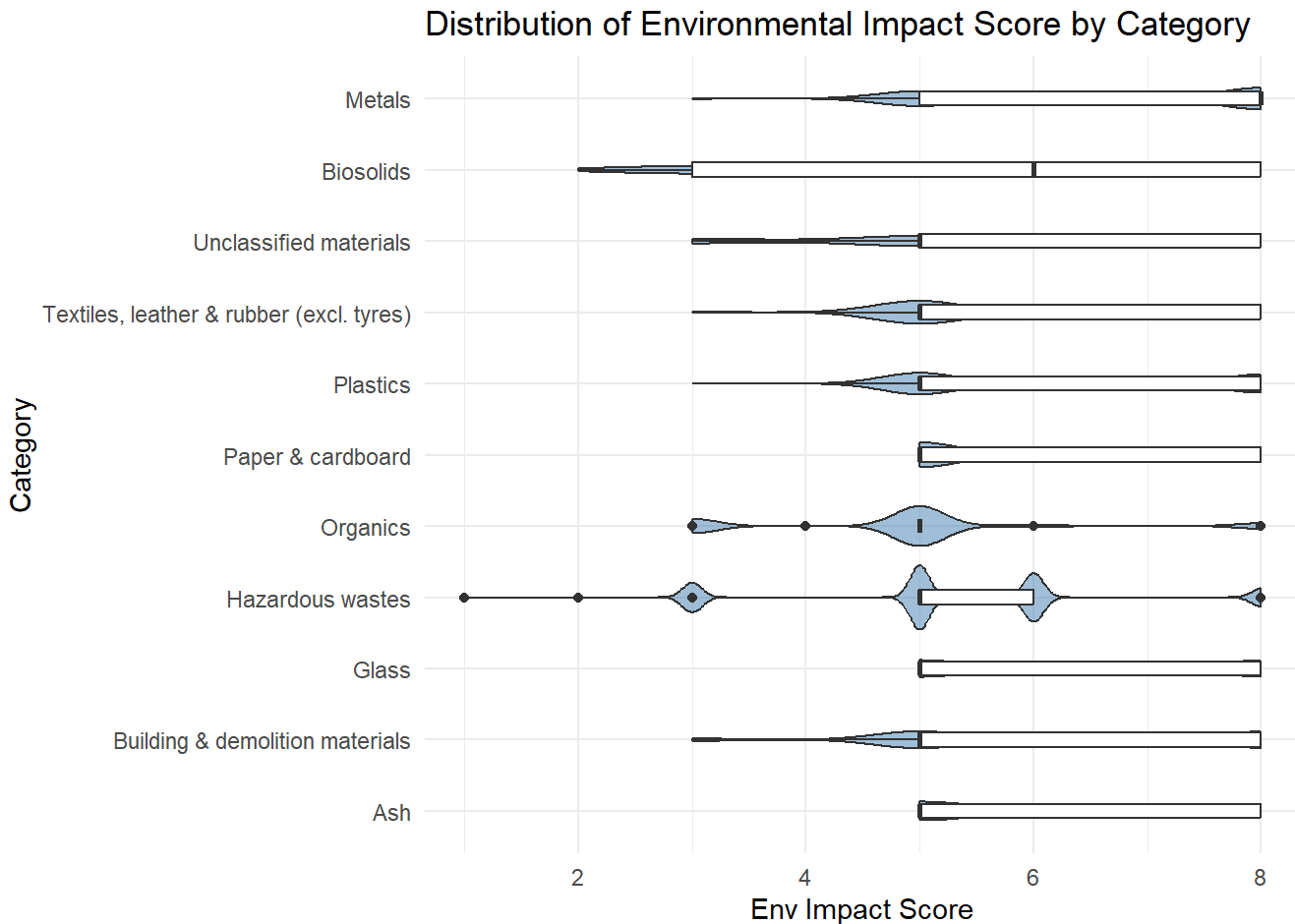
Category <chr>	Mean_... <dbl>	SD_EIS <dbl>	Total_Count <int>	M. <dbl>	First_Q <dbl>
Organics	4.854498	1.316472	4213	3	
Hazardous wastes	5.272445	1.408372	28057	1	
Unclassified materials	5.532847	1.753363	137	3	
Biosolids	5.619792	2.300973	192	2	
Textiles, leather & rubber (excl. tyres)	5.955145	1.451361	379	3	
Ash	6.032000	1.430858	125	5	
Paper & cardboard	6.125000	1.453413	696	5	
Building & demolition materials	6.223060	1.675648	928	3	
Plastics	6.360465	1.466407	946	3	
Glass	6.456311	1.501796	309	5	

1-10 of 11 rows | 1-7 of 9 columns

Previous 1 2 Next

From the category_summary_table above, we realised some category has same min, first quartile, median, third quartile or max falling on the same count. If a boxplot is used itslef will collapse into a flat line, visually uninformative since it hides how many observation at the value. Hence, we visualize them using a boxplot with violin plot to ensure count can be reflected at region of collapse flat line of boxplot, with x-axis as Mean_EIS and y-axis as category.

```
ggplot(waste_year_formatted,
      aes(x = reorder(Category, Env_Impact_Score, median), y = Env_Impact_Score)) +
  geom_violin(fill = "steelblue", alpha = 0.5) +
  geom_boxplot(width = 0.2, fill = "white") +
  coord_flip() +
  labs(
    title = "Distribution of Environmental Impact Score by Category",
    x = "Category",
    y = "Env Impact Score"
  ) +
  theme_minimal()
```



From the category_summary_table and plot, we observed that:

1. Metals has highest Mean_EIS of 6.697350. The boxplot shows Metals waste are more right skewed, meaning its waste process positively affects the environment.
2. Others category such as Ash, Paper & cardboard, Building & demolition materials and Plastics also have relatively high mean above 6 and right skewed.
3. Organics has lowest Mean_EIS of 4.854498. The boxplot is entirely invisible because its first quartile, median and third quartile falls under the same value, indicating Organics is less spread but instead heavily concentrate at a single value of 5.
4. Overall, the plots suggest that Category of waste does influence the EIS. But we also need to consider that each category group size are very contrast, with highest count recording 28057 for hazardous waste and lowest 125 for Ash.

Is Stream a factor?

To analyse if Stream influence EIS, we will observe the statistical information of each Stream.

```
stream_summary_table <- waste_year_formatted %>%
  group_by(Stream) %>%
  summarise(
    Mean_EIS = mean(Env_Impact_Score, na.rm = TRUE),
    SD_EIS = sd(Env_Impact_Score, na.rm = TRUE),
    Total_Count = n(),
    Min = min(Env_Impact_Score, na.rm = TRUE),
    First_Quartile = quantile(Env_Impact_Score, 0.25, na.rm = TRUE),
    Median = median(Env_Impact_Score, na.rm = TRUE),
    Third_Quartile = quantile(Env_Impact_Score, 0.75, na.rm = TRUE),
    Max = max(Env_Impact_Score, na.rm = TRUE)
  )

stream_summary_table %>% arrange(Mean_EIS)
```

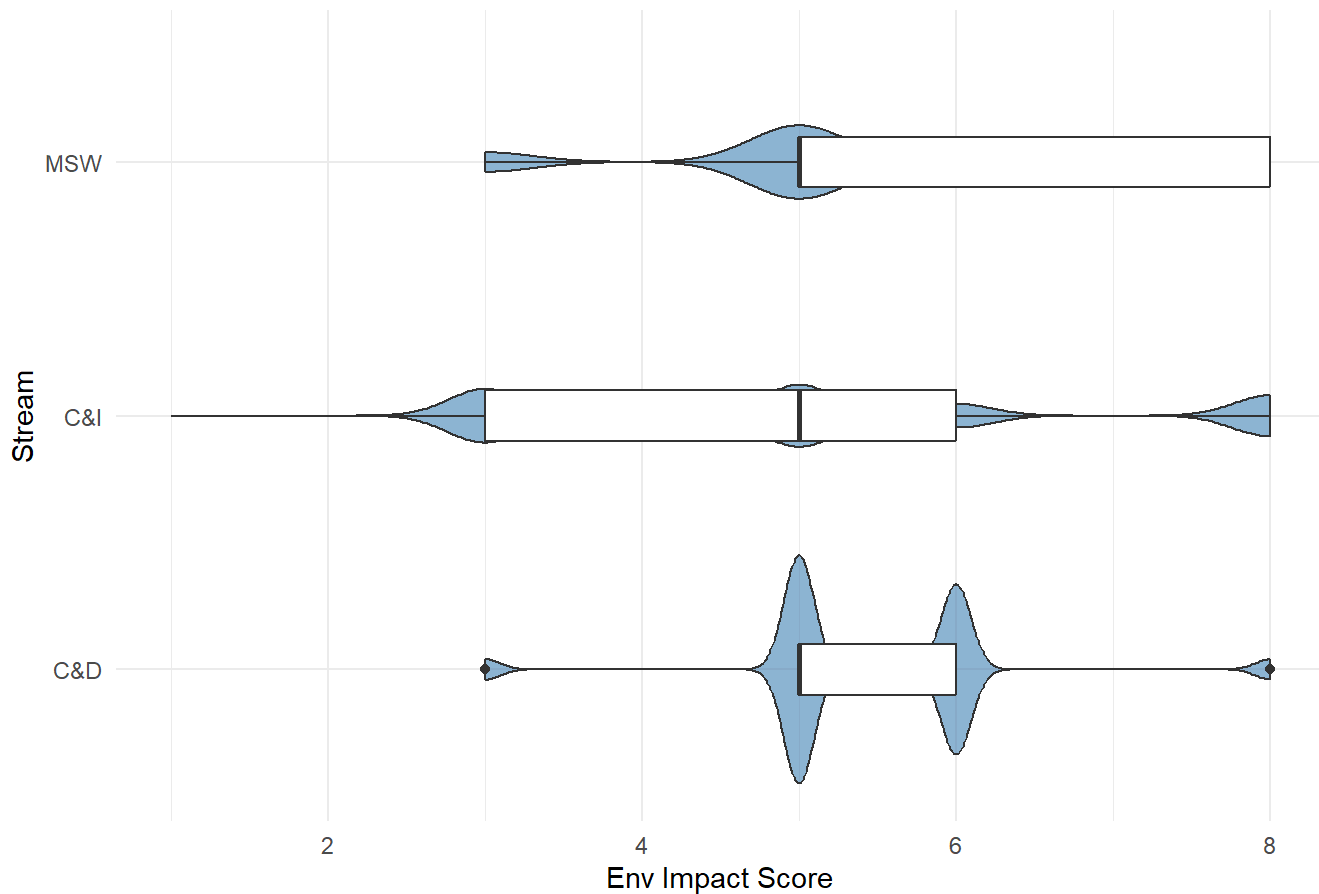
Stre...	Mean_...	SD_EIS	Total_Count	M..	First_Quartile	Medi...	Third_Quartile	M..
<chr>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
C&I	5.216365	1.824120	17207	1	3	5	6	8
C&D	5.429679	0.894584	16773	3	5	5	6	8
MSW	5.609047	1.697877	2719	3	5	5	8	8

3 rows

It is observed that their Mean_EIS is very closed to one another with C&I with 5.216365, C&D with 5.429679 and MSW with 5.609047. Something in common is all 3 stream have the same median of 5, suggesting a violinplot boxplot will be optimal as it shows similar bimodal shapes to visualise how frequency is spread across all 3 streams.

```
ggplot(waste_year_formatted, aes(x = reorder(Stream, Env_Impact_Score, median), y = Env_Impact_Score)) +
  geom_violin(fill = "steelblue", alpha = 0.6) +
  geom_boxplot(width = 0.2, fill = "white") +
  coord_flip() +
  labs(
    title = "Distribution of Env Impact Score by Stream",
    x = "Stream",
    y = "Env Impact Score"
  ) +
  theme_minimal()
```

Distribution of Env Impact Score by Stream



From the stream_summary_table and boxplot violinplot, we observe that:

1. Huge proportion of MSW waste receive EIS of 5 and above as density is heavier at the range of EIS ≥ 5 .
2. C&I is distributed across EIS of 3, 5, 6 and 8, with a median of 5. A wider spread is observed at lower EIS (< 5) compared to higher EIS (> 5).
3. C&D has majority distribution of EIS = 5 and 6.
4. Overall, the mean of all 3 Stream do not differ significantly, it suggest that Stream alone is not very influential over any variation on EIS.

Is Fate a factor?

To analyse if Fate influence EIS, we will observe the statistical information of Fate.

```
summary_fate_table <- waste_year_formatted %>%
  group_by(Fate) %>%
  summarise(
    Mean_EIS = mean(Env_Impact_Score, na.rm = TRUE),
    SD_EIS = sd(Env_Impact_Score, na.rm = TRUE),
    Total_Count = n(),
    Min = min(Env_Impact_Score, na.rm = TRUE),
    First_Quartile = quantile(Env_Impact_Score, 0.25, na.rm = TRUE),
    Median = median(Env_Impact_Score, na.rm = TRUE),
    Third_Quartile = quantile(Env_Impact_Score, 0.75, na.rm = TRUE),
    Max = max(Env_Impact_Score, na.rm = TRUE)
  ) %>% arrange(Mean_EIS)
```

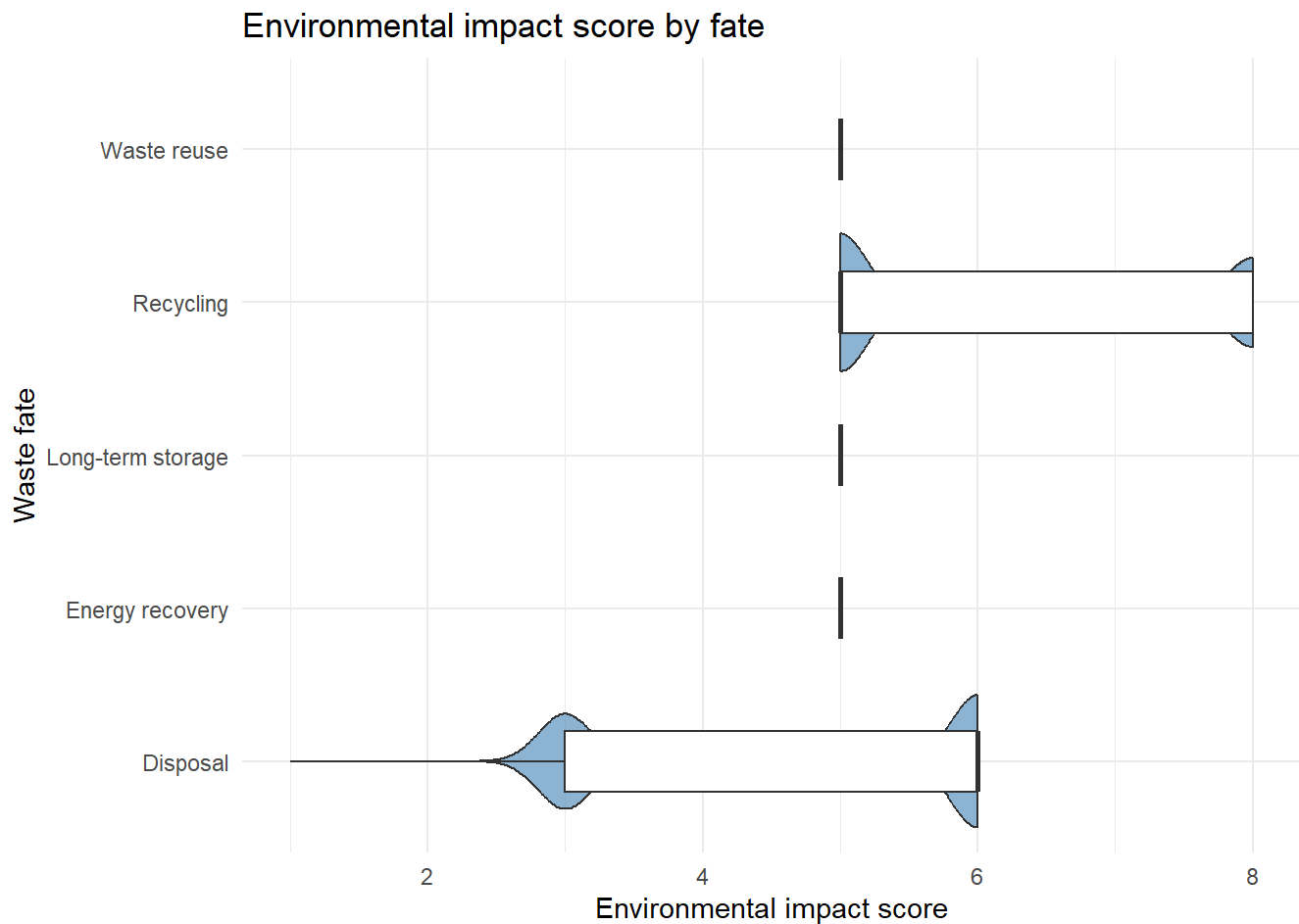
```
summary_fate_table
```

Fate <chr>	Mean_... <dbl>	SD_EIS <dbl>	Total_Count <int>	M. <dbl>	First_Quartile <dbl>	Med... <dbl>	Third_Quan <dbl>
Disposal	4.745634	1.480918	15116	1	3	6	
Energy recovery	5.000000	0.000000	7570	5	5	5	
Long-term storage	5.000000	0.000000	60	5	5	5	
Waste reuse	5.000000	0.000000	19	5	5	5	
Recycling	6.179202	1.465347	13934	5	5	5	

5 rows

From the summary_fate_table above, we observe that Energy recovery, Long-term storage and Waste reuse has standard deviation of 0, meaning all data points in these set are identical and does not spread out of the Mean_EIS of 5.

```
ggplot(
  waste_year_formatted,
  aes(
    x = Fate,
    y = Env_Impact_Score
  )
) +
  geom_violin(fill = "steelblue", alpha = 0.6) +
  geom_boxplot(width = 0.4, fill = "white") +
  coord_flip() +
  labs(
    title = "Environmental impact score by fate",
    x = "Waste fate",
    y = "Environmental impact score"
  ) +
  theme_minimal()
```



From the fate_summary_table and boxplot violinplot, we observe that:

1. Recycling has highest Mean_EIS of 6.179202 with variation distributed at 5 and 8. This signifies the waste process that results in recycling fate is very likely to positively affects the environment.
2. Energy recovery, Long-term storage and Waste with Mean_EIS of 5 have boxplot collapsed into a single line, meaning they concentrate completely at 5
3. Disposal has lowest Mean_EIS of 4.745634 even though its median is 6, which is the highest from all 5 fates. This is because majority of disposal receive an EIS below the median, resulting it to be left-skewed.
4. Overall, Fate is a factor influencing EIS. But we need to consider that the current dataset is underrepresented for Long-term storage and Waste reuse, hence would require a more careful interpretation.

Is Core_Non-core a factor?

To analyse if Core_Non-core influence EIS, we will observe the statistical information of Core_Non-core.

```
core_summary_table <- waste_year_formatted %>%
  group_by(`Core_Non-core`) %>%
  summarise(
    Mean_EIS = mean(Env_Impact_Score, na.rm = TRUE),
    SD_EIS = sd(Env_Impact_Score, na.rm = TRUE),
    Total_Count = n(),
    Min = min(Env_Impact_Score, na.rm = TRUE),
    First_Quartile = quantile(Env_Impact_Score, 0.25, na.rm = TRUE),
    Median = median(Env_Impact_Score, na.rm = TRUE),
    Third_Quartile = quantile(Env_Impact_Score, 0.75, na.rm = TRUE),
    Max = max(Env_Impact_Score, na.rm = TRUE),
    Mean = mean(Env_Impact_Score)
  ) %>% arrange(Mean_EIS)

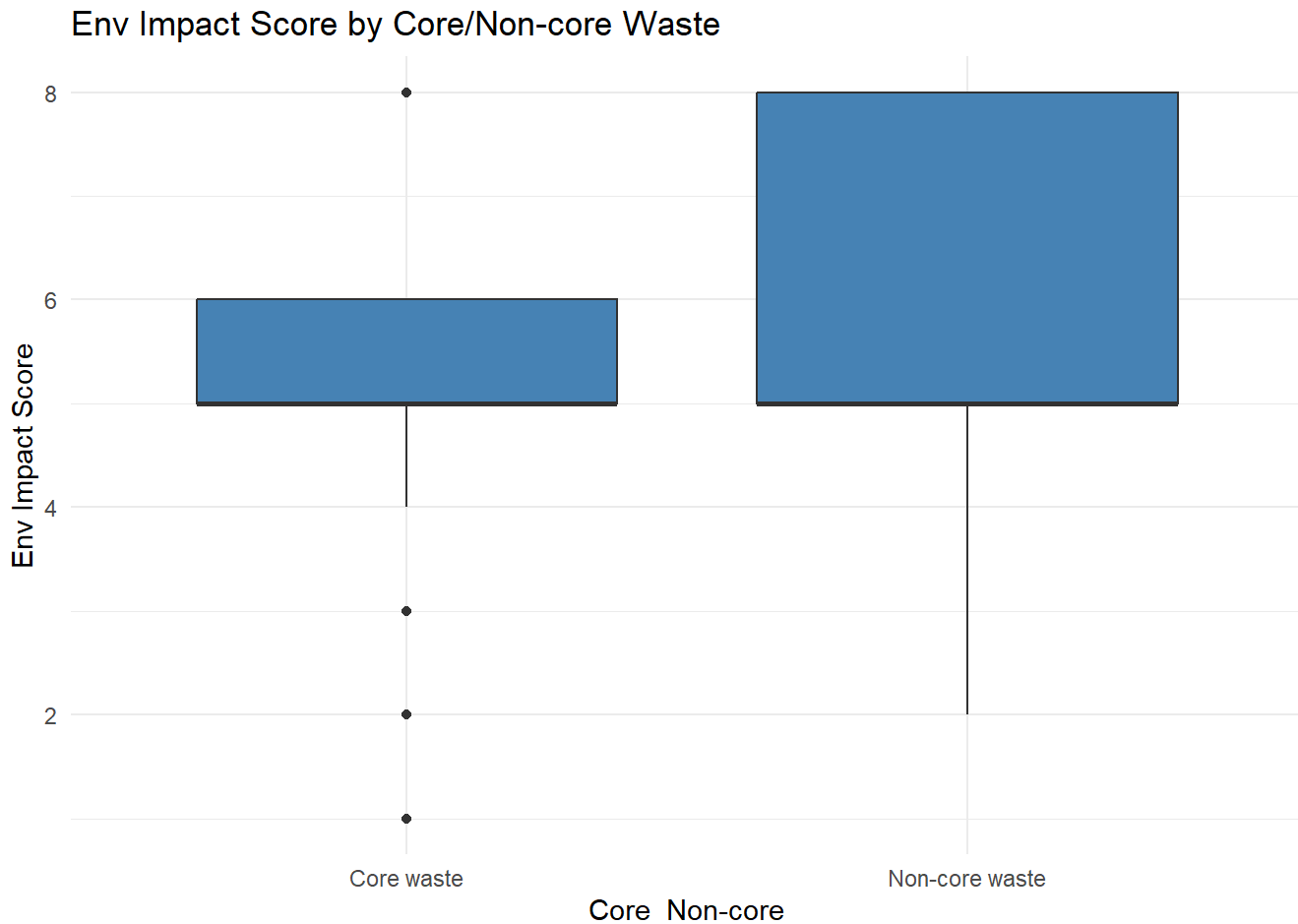
core_summary_table
```

Core_Non-core <chr>	Mean_... <dbl>	SD_EIS <dbl>	Total_Count <int>	M. <dbl>	First_Quartile <dbl>	Med... <dbl>	Third_Quartile <dbl>
Core waste	5.340673	1.467687	36560	1	5	5	6
Non-core waste	5.942446	1.478198	139	2	5	5	8

2 rows

To visualise the distribution, a boxplot is created.

```
ggplot(waste_year_formatted, aes(x = `Core_Non-core`, y = Env_Impact_Score)) +
  geom_boxplot(fill = "steelblue") +
  labs(
    title = "Env Impact Score by Core/Non-core Waste",
    x = "Core_Non-core",
    y = "Env Impact Score"
  ) +
  theme_minimal()
```



From core_summary_table and boxplot, we observe:

1. The mean for Non-core waste (5.942446) is slightly higher than Core-waste (5.340673) although both have same median of 5.
2. Both Core waste and Non-core waste are right skewed. Non-core shows a stronger degree of positive skew than Core.
3. A contrasting observation is total count of Non-core waste is very low compared to Core waste, indicating the dataset is lacking in Non-core waste entries.
4. Overall, due to the contrasting in count of the 2 core-non core category, the difference in mean itself is not strong enough to justify Core_Non-core as a factor on EIS.

Is economic growth a factor?

To analyse, we first create a new state_econ_score_summary table that computes the Mean_EIS, grouped by state and year.

```
state_econ_score_summary <- waste_year_formatted %>%
  group_by(Year, State, Economic_Growth) %>%
  summarise(
    # mean EIS by state and year
    Mean_EIS = mean(Env_Impact_Score, na.rm = TRUE),
    Count = n()
  )
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Year, State, and Economic_Growth.
## i Output is grouped by Year and State.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Year, State, Economic_Growth))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.
```

```
state_econ_score_summary
```

Year	State	Economic_Growth	Mean_EIS	Count
<dbl>	<chr>	<dbl>	<dbl>	<int>
2006	ACT	2.88	5.089744	312
2006	NSW	2.08	5.330909	275
2006	NT	1.41	5.281787	291
2006	QLD	4.36	5.106897	290
2006	SA	1.69	5.175793	347
2006	TAS	2.72	5.228669	293
2006	VIC	1.35	5.546012	326
2006	WA	3.34	5.191429	350
2007	ACT	2.43	5.235808	229
2007	NSW	2.61	5.379913	229

1-10 of 120 rows

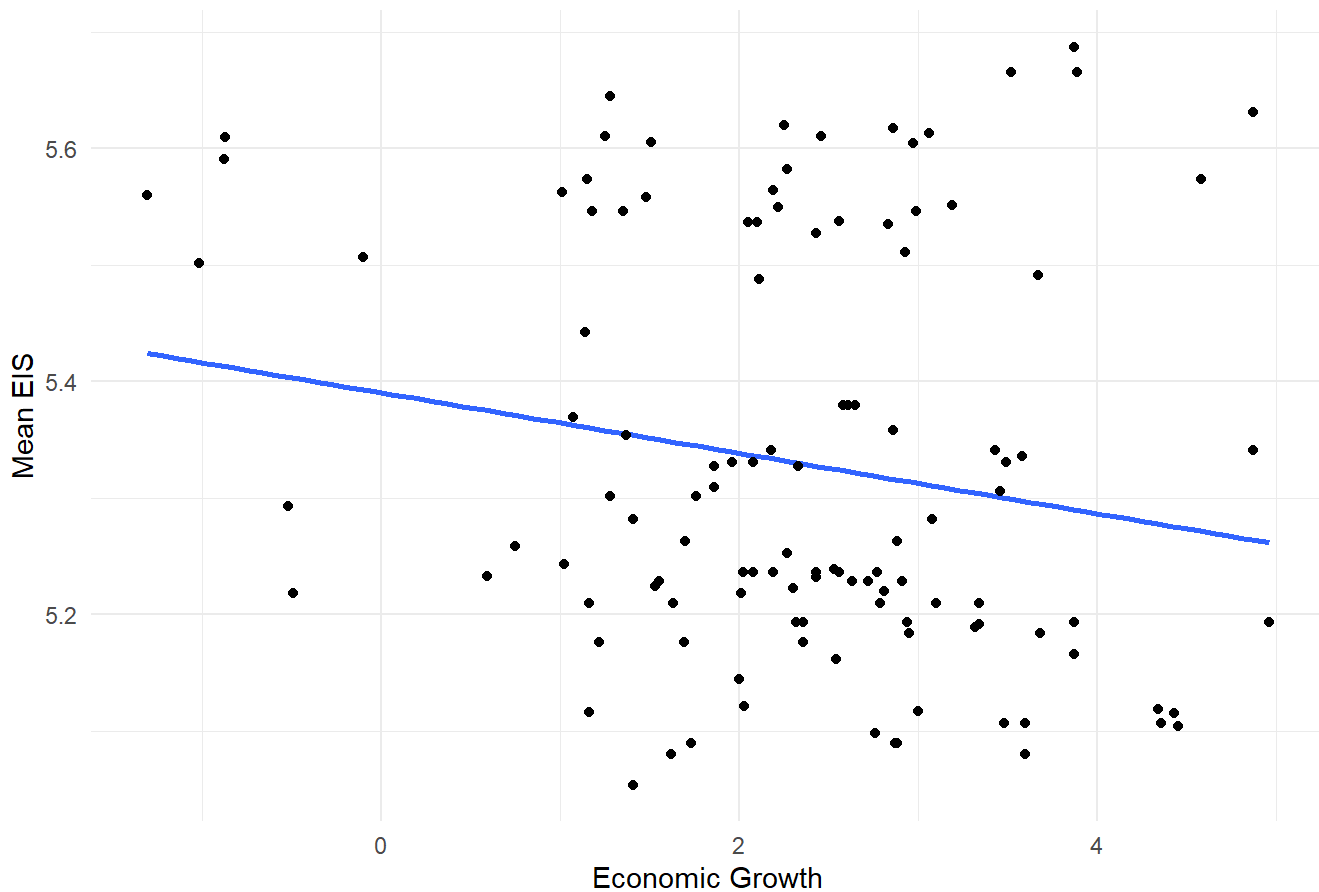
Previous 1 2 3 4 5 6 ... 12 Next

We plot a scatterplot with a linear regression line to show the relationship between EIS and economic growth (x-axis as economic growth percentage and y-axis as mean EIS)

```
ggplot(state_econ_score_summary,
       aes(x = Economic_Growth,
           y = Mean_EIS)) +
  geom_smooth(method = "lm", se = FALSE) +
  geom_point() +
  labs(
    title = "Mean Environmental Impact Score against Economic Growth",
    x = "Economic Growth",
    y = "Mean EIS"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Mean Environmental Impact Score against Economic Growth



Here, we observed that:

- Shows a negative trend, indicating as economic growth increases, mean EIS decreases.
- However, the scattered dots does not show a clear trend as the fitted trendline as dots are dispersed randomly all across the plot.

To analyse the strength of correlation between mean EIS and economic growth, we carry out a Pearson correlation test since both economic growth and mean EIS are both continuous quantitative variables.

```
r <- cor(state_econ_score_summary$Economic_Growth, state_econ_score_summary$Mean_EIS, method = "pearson")
r
```

```
## [1] -0.1774929
```

The r value obtained is negative (-0.1774929), indicating a weak correlation between mean EIS and economic growth. Therefore, economic growth per state-year is not as influential on environmental impact score compared to previously analysed factor (Fate, Stream, Category).

Overall model fit

```
model <- lm(  
  Env_Impact_Score ~  
    Fate +  
    Stream +  
    Category +  
    `Core_Non-core`,  
  data = waste_year_formatted  
)  
  
summary(model)
```

```
##
## Call:
## lm(formula = Env_Impact_Score ~ Fate + Stream + Category + `Core_Non-core`,
##     data = waste_year_formatted)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.6573 -1.2377  0.0056  1.1207  2.3505
##
## Coefficients:
##                                     Estimate Std. Error t value
## (Intercept)                      6.06716     0.38376  15.810
## FateEnergy recovery                0.33710     0.01898   17.765
## FateLong-term storage             -0.73525     0.23406   -3.141
## FateRecycling                     1.35840     0.01655  82.064
## FateWaste reuse                   -0.05915     0.30282   -0.195
## StreamC&I                       -0.22203     0.01415 -15.695
## StreamMSW                       -0.03744     0.03116   -1.202
## CategoryBiosolids                 -0.90454     0.39451   -2.293
## CategoryBuilding & demolition materials -0.99392     0.38560   -2.578
## CategoryGlass                     -0.88276     0.39039   -2.261
## CategoryHazardous wastes          -1.18784     0.38329   -3.099
## CategoryMetals                    -0.60139     0.38637   -1.557
## CategoryOrganics                  -1.55399     0.38415   -4.045
## CategoryPaper & cardboard          -1.21133     0.38646   -3.134
## CategoryPlastics                  -0.91235     0.38564   -2.366
## CategoryTextiles, leather & rubber (excl. tyres) -1.35227     0.38909   -3.475
## CategoryUnclassified materials    -1.32830     0.39862   -3.332
## `Core_Non-core`Non-core waste     -0.10988     0.34692   -0.317
##
##                                     Pr(>|t|)
## (Intercept)                       < 2e-16 ***
## FateEnergy recovery                 < 2e-16 ***
## FateLong-term storage               0.001683 **
## FateRecycling                       < 2e-16 ***
## FateWaste reuse                     0.845142
## StreamC&I                          < 2e-16 ***
## StreamMSW                          0.229495
## CategoryBiosolids                   0.021864 *
## CategoryBuilding & demolition materials 0.009953 **
## CategoryGlass                       0.023750 *
## CategoryHazardous wastes             0.001943 **
## CategoryMetals                       0.119591
## CategoryOrganics                     5.24e-05 ***
## CategoryPaper & cardboard            0.001723 **
## CategoryPlastics                     0.017996 *
## CategoryTextiles, leather & rubber (excl. tyres) 0.000511 ***
## CategoryUnclassified materials       0.000862 ***
## `Core_Non-core`Non-core waste        0.751450
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.298 on 36681 degrees of freedom
```

```
## Multiple R-squared:  0.2191, Adjusted R-squared:  0.2187
## F-statistic: 605.3 on 17 and 36681 DF,  p-value: < 2.2e-16
```

Based on the missing output drop as the implicit baseline:

- Fate baseline = Disposal
- Stream baseline = C&D
- Category baseline = Ash
- Core_Non-core baseline = Core waste
- State baseline = ACT

Whole model together explains 23.2% of variance in EIS. Hence, even though aspect (Fate, Stream, Category, Core_Non-core, Economic growth) have some relationship with EIS, a great portion of the EIS is still unexplained by these aspect.

Overall it shows factor-to-factor interpretation:

1. Fate is the strongest factor on EIS after controlling other variable. Recycling is the biggest driver to a more positive EIS, followed by energy recover, long-term storage, waste reuse and disposal (baseline).
2. Stream has the second strongest factor on EIS after controlling other variable, with C&I leading to more positive EIS, followed by MSW and C&D (baseline).
3. Category is next. Several categories remain significant even after controlling other variable. For example, Organics, Unclassified materials, Textiles, Building & demolition, Hazardous wastes, Paper & cardboard, Biosolids. But a few are no longer significant, such as Metals, Plastics and Glass. This informs that category univariate effect was not purely a proxy for Fate, some categories have their own independent association with EIS, but not all of them.
4. Core_Non-core is the most insignificant factor on EIS, even after controlling other variables.

Q8. Disposal size variation and disposal frequency of each state

In previous task, Tonnes in waste_year_formatted is already cleaned. Hence, no additional data cleaning required.

Observe tyres disposal

First, we observe waste entries with Type of "Tyres (T140)" and Fate of "Disposal". We need to observe the number of entries and Tonnes values to determine a suitable tonnes_range.

```
tyre_disposal <- waste_year_formatted %>%
  filter(
    Type == "Tyres (T140)",
    Fate == "Disposal",
  ) %>%
  arrange()

tyre_disposal %>% select(Type, Fate, Tonnes) %>% arrange(Tonnes)
```

Type <chr>	Fate <chr>	Tonnes <dbl>
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
Tyres (T140)	Disposal	0.00000
1-10 of 718 rows		Previous 1 2 3 4 5 6 ... 72 Next

We consider disposal with tonnes > 0 as actual meaningful disposal, thus, we will drop of tyres disposal with zero-tonnes for analysis.

```
tyre_disposal <- tyre_disposal %>% filter(Tonnes > 0)

tyre_disposal %>% select(Type, Fate, Tonnes) %>% arrange(Tonnes)
```

Type <chr>	Fate <chr>	Tonnes <dbl>
Tyres (T140)	Disposal	86.00323
Tyres (T140)	Disposal	95.86016
Tyres (T140)	Disposal	100.91743
Tyres (T140)	Disposal	100.91743
Tyres (T140)	Disposal	101.85185
Tyres (T140)	Disposal	101.85185
Tyres (T140)	Disposal	102.76265
Tyres (T140)	Disposal	103.74809
Tyres (T140)	Disposal	106.52174
Tyres (T140)	Disposal	106.52174
1-10 of 152 rows		Previous 1 2 3 4 5 6 ... 16 Next

Categorization of Tonnes range

Now, we check the summary of Tonnes column in tyre_disposal (min, q1, median, q3, max)

```
summary(tyre_disposal$Tonnes)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      86    2088    6211    21201   45607    70411
```

We observe that median (6211) is lower than mean (21201), closer to first quartile (2088) compared to third quartile (45607). Thus, a euqal-interval binning appraoch is unsuitable. Instead, we will use a non-linear interval for determing tonnes_range variable.

To decide on range, we first observe some sensible breakpoints:

```
tyre_disposal %>%
  filter(Tonnes > 0) %>%
  pull(Tonnes) %>%
  quantile(probs = c(
    0,
    0.25,
    0.5,
    0.75,
    0.9,
    0.95,
    1
  ))
```

```
##           0%          25%          50%          75%          90%          95%
##    86.00323  2087.94381  6211.20817  45606.64815  60859.89993  64227.91414
##           100%
## 70411.02289
```

Now, we can decide on Tonnes_ranges with values:

- 0-500
- 500-5000
- 5000-10000
- 10000-50000
- 50000+

We update the tyre_disposal table by mutating a new Tonnes_range column and apply case_when to determine each waste Tonnes_range category.

```

tyre_disposal <- tyre_disposal %>%
  mutate(
    Tonnes_Range = case_when(
      Tonnes > 0 & Tonnes <= 500 ~ "0-500",
      Tonnes > 500 & Tonnes <= 5000 ~ "500-5000",
      Tonnes > 5000 & Tonnes <= 10000 ~ "5000-10000",
      Tonnes > 10000 & Tonnes <= 50000 ~ "10000-50000",
      Tonnes > 50000 ~ "50000+"
    )
  )

tyre_disposal

```

Case...	Year_State_ID	Category	Type	Stre...	Fate	Tonnes	Core_
<dbl>	<dbl>	<chr>	<chr>	<chr>	<chr>	<dbl>	<chr>
35082	1001	Hazardous wastes	Tyres (T140)	C&I	Disposal	1452.47325	Core v
35083	1001	Hazardous wastes	Tyres (T140)	C&I	Disposal	103.74809	Core v
35831	1003	Hazardous wastes	Tyres (T140)	C&I	Disposal	45684.27893	Core v
35832	1003	Hazardous wastes	Tyres (T140)	C&I	Disposal	2643.76614	Core v
36372	1004	Hazardous wastes	Tyres (T140)	C&I	Disposal	1816.78908	Core v
36373	1004	Hazardous wastes	Tyres (T140)	C&I	Disposal	106.86995	Core v
36903	1005	Hazardous wastes	Tyres (T140)	C&I	Disposal	29663.47410	Core v
36904	1005	Hazardous wastes	Tyres (T140)	C&I	Disposal	1555.77661	Core v
37459	1006	Hazardous wastes	Tyres (T140)	C&I	Disposal	6857.59463	Core v
37460	1006	Hazardous wastes	Tyres (T140)	C&I	Disposal	415.61180	Core v

1-10 of 152 rows | 1-8 of 16 columns

Previous 1 2 3 4 5 6 ... 16 Next

Now that tyre_disposal table has each waste categorised into respective Tonnes_range, we can create a new tonnes_state table that group by each State and Tonnes_range to obtain the frequency of each Tonnes_Range by state.

```

tonnes_state <- tyre_disposal %>%
  group_by(State, Tonnes_Range) %>%
  summarise(
    Frequency = n()
  ) %>%
  ungroup()

```

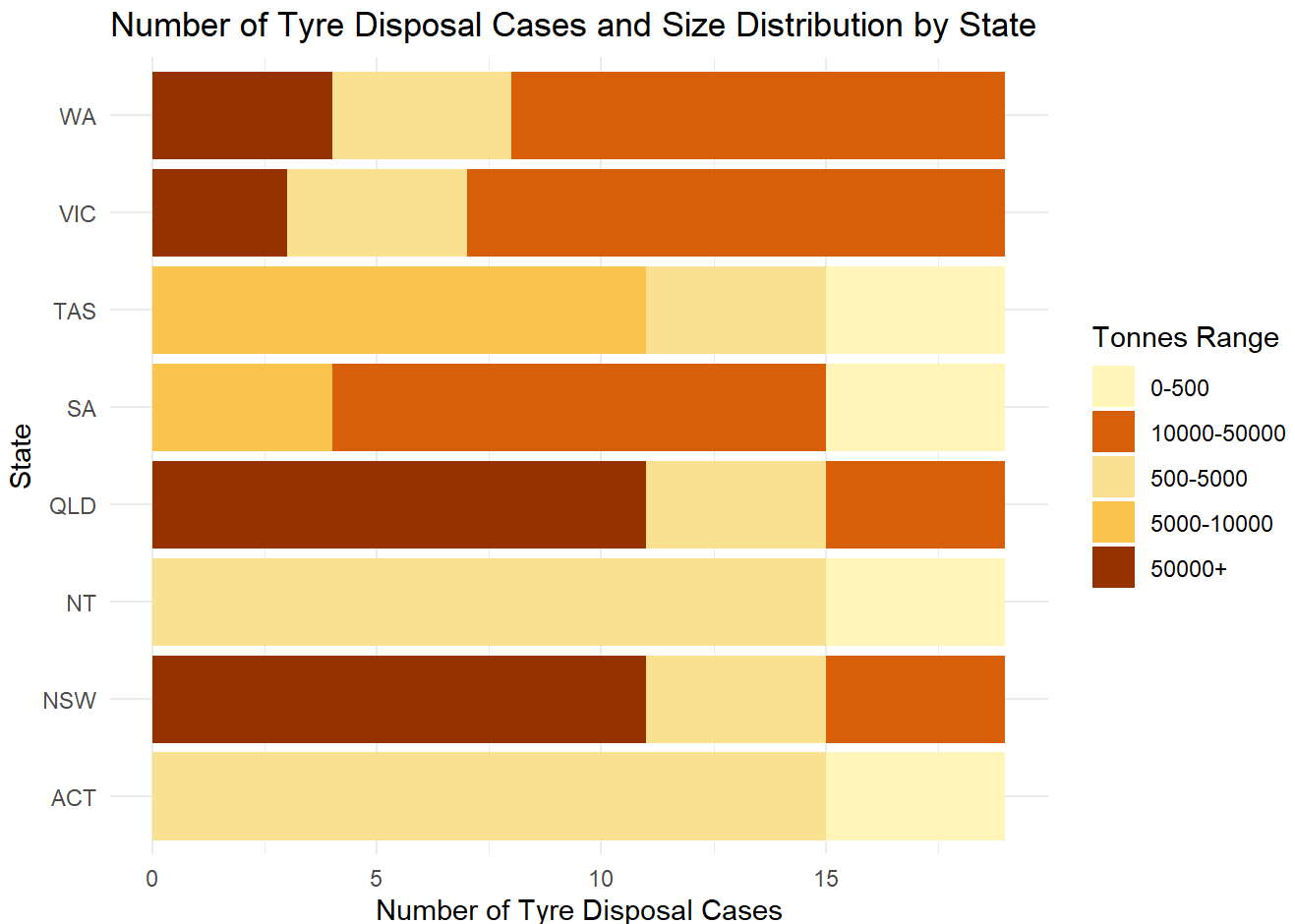
```
## `summarise()` has regrouped the output.  
## i Summaries were computed grouped by State and Tonnes_Range.  
## i Output is grouped by State.  
## i Use `summarise(.groups = "drop_last")` to silence this message.  
## i Use `summarise(.by = c(State, Tonnes_Range))` for per-operation grouping  
##   (`?dplyr::dplyr_by`) instead.
```

tonnes_state

State <chr>	Tonnes_Range <chr>	Frequency <int>
ACT	0-500	4
ACT	500-5000	15
NSW	10000-50000	4
NSW	500-5000	4
NSW	50000+	11
NT	0-500	4
NT	500-5000	15
QLD	10000-50000	4
QLD	500-5000	4
QLD	50000+	11
1-10 of 22 rows		Previous 1 2 3 Next

To visualize the frequency of each Tonnes_range by state, a stacked barchart is used. A bar chart is used over boxplot or histogram so that manger can compare categorical states against each other on 2 aspect (tyre disposal frequency and tonnes range). Stacking bars show both aspect without requiring manager to read multiple distribution plots per state.

```
ggplot(tonnes_state, aes(x = reorder(State, Frequency, sum), y = Frequency, fill = Tonnes_Range)) +
  geom_col(position = "stack") +
  coord_flip() +
  scale_fill_manual(values = c(
    "0-500" = "#FFF7BC", # light shade for less significant disposal
    "500-5000" = "#FEE391",
    "5000-10000" = "#FEC44F",
    "10000-50000" = "#D95F0E",
    "50000+" = "#993404" # darker shade to highlight highest tonnes range
  )) +
  labs(
    title = "Number of Tyre Disposal Cases and Size Distribution by State",
    x = "State",
    y = "Number of Tyre Disposal Cases",
    fill = "Tonnes Range"
  ) +
  theme_minimal()
```



From the bar chart, it tells the manager:

1. State with higher number of disposal cases represented by darker shade (eg: 50000+) are great state for investing in large-scale processing capacity. It show a pattern of large, sustained disposal events, meaning large-scale processing infrastructure built there is more likely to have a consistent supply of substantial waste volumes to process.

2. State with disposal cases higher number of disposal cases represented by lighter bins (eg: 0-500, 500-5000) suggest many small disposal event, justifying it is not suitable for large infrastructure investment in these states. Large-scale infrastructure being built there risks being underutilized, since there is no consistent stream of high-volume disposal to justify.
3. VIC, NSW, QLD and WA are best candidates state to invest for large-scale tyres processing capacity due to higher volume of disposal event happening at range of 50000+. This suggestion is based on disposal volume and consistency alone. The final investment decisions would also need to weight factors outside this dataset, such as transport, logistics, existing infrastructure and labour costs.

Q9 Factor influencing yearly trend of total C&D waste tones

External dataset from ABS_Population.xlsx

Australia Bureau of Statistics, Population Size and Growth:
<https://www.abs.gov.au/statistics/people/population/historical-population/latest-release#data-downloads>
(<https://www.abs.gov.au/statistics/people/population/historical-population/latest-release#data-downloads>)

```
# dataset is downloaded and renamed to ABS_Population.xlsx
# read from ABS_Population.xlsx, skip the first 4 row (headers)
population <- read_excel("data/ABS_Population.xlsx", skip = 4)
population
```

Sex
<chr>
Male
Male
Male
Male
Male
Male
Male
Male
Male
Female

1-10 of 36 rows | 1-1 of 122 columns

Previous1234Next

Data cleaning external dataset

We observed the original dataset consist on each year from 1902-2021 as individual column. We filter only relevant years, which are 2006-2020, matching the Waste.csv and Year_State_ID.csv.

```
population <- population %>%
  select(Sex, `State/Territory`, `2006`:`2020`) %>%
  arrange(`State/Territory`)
```

population

Sex

<chr>

Male

Female

Person

Male

Female

Person

Male

Female

Person

Male

1-10 of 36 rows | 1-1 of 17 columns

Previous 1 2 3 4 Next

We have some info about the State/Territory value.

1. State/Territory of "Australia(e)" refers the the total amount of Male, Female and overall for each year across the entire nation.
2. State/Territory with NA have entire rows being NA, except Sex, which indicates the NA is a result from header.
3. State/Territory have mixture of upper and lowercase.

We can replace the State/Territory values to match our waste_year_formatted.

```
# view how state are formatted in waste_year_formatted
waste_year_formatted %>% distinct(State)
```

State

<chr>

ACT

State
<chr>
NSW
NT
QLD
SA
TAS
VIC
WA
8 rows

```
# view how State/Territory are formatted in population
population %>% distinct(`State/Territory`)
```

State/Territory
<chr>
ACT(d)
Australia(e)
NSW
NT
Qld
SA
Tas.
Vic.
WA
NA
1-10 of 10 rows

We capitalized and abbreviate all State/Territory values in population using recode(), replace “Australia(e)” with a meaningful name using case_when() and finally drop off all NA values .

```
population_formatted <- population %>%
  mutate(
    `State/Territory` = recode(
      `State/Territory`,
      "Vic." = "VIC",
      "Qld" = "QLD",
      "Tas." = "TAS",
      "ACT(d)" = "ACT"
    ),
    `State/Territory` = case_when(
      `State/Territory` == "Australia(e)" & Sex == "Male" ~ "Total Male",
      `State/Territory` == "Australia(e)" & Sex == "Female" ~ "Total Female",
      `State/Territory` == "Australia(e)" & Sex == "Person" ~ "Total Population",
      TRUE ~ `State/Territory`
    )
  ) %>%
  filter(!is.na(`State/Territory`)) %>%
  arrange(`State/Territory`)
```

Now, observe `population_formatted` and `waste_year_formatted` State or State/Territory have been standardized, with `population_formatted` having additional "Total Female", "Total Male" and "Total Population".

```
# view how state are formatted in waste_year_formatted
waste_year_formatted %>% distinct(State)
```

State

<chr>

ACT

NSW

NT

QLD

SA

TAS

VIC

WA

8 rows

```
# view how State/Territory are formatted in population_formatted with additional "Total Female",
"Total Male" and "Total Population"
population_formatted %>% distinct(`State/Territory`)
```

State/Territory	
<chr>	
ACT	
NSW	
NT	
QLD	
SA	
TAS	
Total Female	
Total Male	
Total Population	
VIC	
1-10 of 11 rows	
Previous 1 2 Next	

Filtering C&D waste from waste_year_formatted

We are only interested in C&D (Construction and Demolition) Stream waste. So we filter and save these waste in cnd_waste table. Since waste_year_formatted have been cleaned in previous tasks, hence no additional cleaning required here.

```
cnd_waste <- waste_year_formatted %>%
  filter(Stream == "C&D") %>%
  select(-c("Description", "start", "end"))

cnd_waste
```

Case_ID	Year_State_ID	Category	
<dbl>	<dbl>	<chr>	
34662	1001	Building & demolition materials	
34666	1001	Building & demolition materials	
34670	1001	Building & demolition materials	
34674	1001	Building & demolition materials	
34678	1001	Building & demolition materials	
34682	1001	Building & demolition materials	
34683	1001	Building & demolition materials	
34688	1001	Glass	
34692	1001	Glass	

Case_ID <dbl>	Year_State_ID <dbl>	Category <chr>										
34696	1001	Hazardous wastes										
1-10 of 10,000 rows 1-3 of 12 columns			Previous	1	2	3	4	5	6	...	1000	Next

Then, we obtain the total C&D tonnes of each state in each year by grouping into State and Year, sum up Tonnes. Save it into a new `cnd_state` table.

```
cnd_state <- cnd_waste %>%
  group_by(State, Year) %>%
  summarise(
    Total_CND_Tonnes = sum(Tonnes)
  )
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by State and Year.
## i Output is grouped by State.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(State, Year))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
cnd_state %>% arrange(Year)
```

State <chr>	Year <dbl>	Total_CND_Tonnes <dbl>
ACT	2006	5512.7109
NSW	2006	1428826.9774
NT	2006	21341.6547
QLD	2006	619591.2570
SA	2006	73189.5196
TAS	2006	5828.9352
VIC	2006	3012370.0414
WA	2006	148253.4112
ACT	2007	193.5988
NSW	2007	446608.0028
1-10 of 120 rows		
Previous 1 2 3 4 5 6 ... 12 Next		

Joining `cnd_state` and `population_formatted`

Before joining the 2 table, we observe the pattern of both tables.

```
population_formatted %>% head()
```

Sex <chr>	State/Territory <chr>	2006 <dbl>	2007 <dbl>	2008 <dbl>	2009 <dbl>	2010 <dbl>	2011 <dbl>	2012 <dbl>	2013 <dbl>
Male	ACT	1982	3826	3067	3618	3535	3136	4160	3211
Female	ACT	1789	3648	2657	2799	3446	3083	4394	3507
Person	ACT	3771	7474	5724	6417	6981	6219	8554	6718
Male	NSW	22541	48103	57959	57839	45171	37002	41291	48010
Female	NSW	26943	43363	51346	52455	45366	37235	44424	51778
Person	NSW	49484	91466	109305	110294	90537	74237	85715	99788

6 rows | 1-10 of 17 columns

We observe in `population_formatted`, we have columns of:

- Sex: Male, Female, Person
- State/Territory: repeating capitalized abbreviated state name for 3 times due to each row representing Male, Female and Person.
- 2006 to 2020: each value indicating total male, total female and total population of the respective year

```
cnd_state %>% head(20)
```

State <chr>	Year <dbl>	Total_CND_Tonnes <dbl>
ACT	2006	5512.7109
ACT	2007	193.5988
ACT	2008	17062.4719
ACT	2009	9893.8673
ACT	2010	13489.5240
ACT	2011	2764.0004
ACT	2012	3414.9000
ACT	2013	9149.1670
ACT	2014	8988.4624
ACT	2015	44805.0864

1-10 of 20 rows

Previous **1** 2 Next

We observe in `cnd_state` we have columns of:

- State: non-repeating capitalized abbreviated state name for each year

- Year: 2006-2020
- Total_CND_Tonnes: sum of tonnes produced by respective state by year

We need to reshape population_formatted to match cnd_waste. We do this by pivot_longer() to make each individual year column as a value under a single Year column. Then pivot_wider() to create new Total_Male, Total_Female and Total_Population columns after renaming.

```
population_reshaped <- population_formatted %>%
  # convert individual 2006-2020 column into single Year column
  pivot_longer(
    cols = `2006`:`2020`,
    names_to = "Year",
    values_to = "Population"
  ) %>%
  # convert to Year to integer
  mutate(
    Year = as.integer(Year)
  ) %>%
  # rename State/Territory for consistency
  rename(State = `State/Territory`) %>%
  # convert Population to individual sex-specific population column
  pivot_wider(
    names_from = Sex,
    values_from = Population
  ) %>%
  rename(
    Total_Male = Male,
    Total_Female = Female,
    Total_Population = Person
  )
```

population_reshaped

State <chr>	Year <int>	Total_Male <dbl>	Total_Female <dbl>	Total_Population <dbl>
ACT	2006	1982	1789	3771
ACT	2007	3826	3648	7474
ACT	2008	3067	2657	5724
ACT	2009	3618	2799	6417
ACT	2010	3535	3446	6981
ACT	2011	3136	3083	6219
ACT	2012	4160	4394	8554
ACT	2013	3211	3507	6718
ACT	2014	2546	2996	5542

State <chr>	Year <int>	Total_Male <dbl>	Total_Female <dbl>	Total_Population <dbl>
ACT	2015	3324	3690	7014

1-10 of 165 rows

Previous 1 2 3 4 5 6 ... 17 Next

Now, we left join `cnd_state` to `population_reshaped` by Year and State.

```
cnd_population <- cnd_state %>%
  left_join(
    population_reshaped,
    by = c("Year", "State")
  )

cnd_population
```

State <chr>	Year <dbl>	Total_CND_Tonnes <dbl>	Total_Male <dbl>	Total_Female <dbl>	Total_Population <dbl>
ACT	2006	5512.7109	1982	1789	3771
ACT	2007	193.5988	3826	3648	7474
ACT	2008	17062.4719	3067	2657	5724
ACT	2009	9893.8673	3618	2799	6417
ACT	2010	13489.5240	3535	3446	6981
ACT	2011	2764.0004	3136	3083	6219
ACT	2012	3414.9000	4160	4394	8554
ACT	2013	9149.1670	3211	3507	6718
ACT	2014	8988.4624	2546	2996	5542
ACT	2015	44805.0864	3324	3690	7014

1-10 of 120 rows

Previous 1 2 3 4 5 6 ... 12 Next

Correlation between total population and total C&D waste tonnes.

Population and C&D tonnes are both continuous variable, hence we will use Pearson correlation coefficient analyse the strength and direction.

```
# Pearson correlation coefficient
r <- cor(cnd_population$Total_Population, cnd_population$Total_CND_Tonnes, method = "pearson")
r
```

```
## [1] 0.6368048
```

```
# coefficient of determination = R^2
r_squared <- r^2
r_squared
```

```
## [1] 0.4055204
```

Pearson correlation analysis yielded a moderate-to-strong positive linear correlation (0.6368048) between total population and C&D waste tonnes. This confirms that regional population scale is a major structural driver of construction waste tonnes. However, because population accounts for approximately 41% of total variance, macro-economic conditions and local waste management policies also play substantial roles in determining annual C&D waste generation.

Visualisation

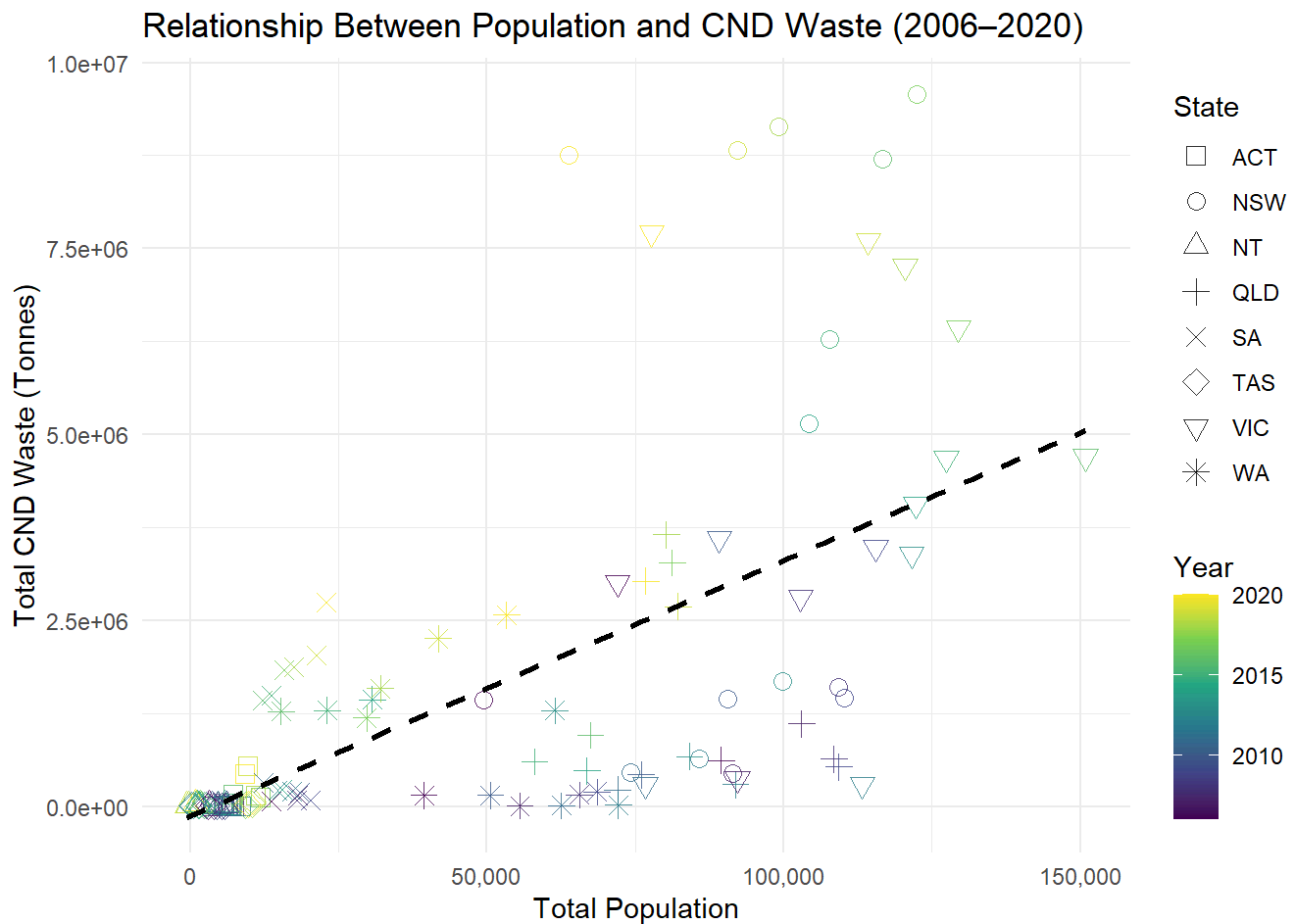
To determine if population size directly affect C&D Tonnes disposal, a scatterplot and linear trendline is used. X-axis represent total population, y-axis represent total C&D tonnes, each point is shaped uniquely according to state.

Scatterplot and trendline is used because:

1. It reveals the relationship, strength and direction between the 2 continuous variables (total population, C&D waste tonnes).
2. Trendline shows the general trend across data points.

```
ggplot(cnd_population,
       aes(x = Total_Population, y = Total_CND_Tonnes)) +
  geom_point(
    aes(color = Year, shape = State), size = 3, alpha = 0.8) +
  geom_smooth(
    method = "lm",
    se = FALSE,
    color = "black",
    linetype = "dashed") +
  scale_color_viridis_c() +
  scale_shape_manual(
    values = c(0, 1, 2, 3, 4, 5, 6, 8)
  ) +
  scale_x_continuous(
    labels = scales::comma
  ) +
  labs(
    title = "Relationship Between Population and CND Waste (2006-2020)",
    x = "Total Population",
    y = "Total CND Waste (Tonnes)",
    color = "Year",
    shape = "State"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



From the scatterplot, we observe that:

- Trendline shows a positive linear relationship between total C&D tonnes and population. States and years with population proportionally generates lower C&D waste tonnes; whereas states and years with higher population increases C&D waste tonnes.
- Generate the hypothesis that the underlying reason can be when population increases, more housing and facility will need to be constructed to support the community, resulting in higher disposal of C&D due to higher construction activity.

Overall, by investigating external dataset on Australia states population across the year, we obtained the result that population has a strong individual relationship to yearly trend of total C&D waste tonnes. This is supported by Pearson correlation analysis and scatterplot.